
TurboSens: An Interactive Turbofan Benchmark for World-Model Evaluation and Physical Understanding

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 World models learn representations of physical systems from observations, but
2 evaluating whether they recover the underlying causal state is difficult when that
3 state is unobserved. We introduce TurboSens, an interactive turbofan degradation
4 simulator designed as an evaluation instrument for world model representations. It
5 provides paired access to sensor streams and the full ground truth health state at
6 every timestep: a ten dimensional latent underdetermined by seven sensors. This
7 enables a direct protocol: train a self-supervised world model on observations
8 alone, then evaluate its representations on state recovery, forecasting, and out of
9 distribution generalization with action conditioning. We release two progressively
10 challenging scenarios (TurboSens1 and TurboSens2) with configurable degradation,
11 maintenance, a two channel hidden state, and a phantom observation map where
12 sensed and true states diverge.

13 The turbofan domain offers complex, industry relevant dynamics: heterogeneous
14 wear, sparse stochastic events, weather, and region specific conditions. Unlike
15 pixel or language benchmarks, TurboSens provides ground truth, allowing us to
16 ask what representations preserve under distribution shift, where failures origi-
17 nate (encoder vs predictor), and how temporal horizons affect robustness. As a
18 demonstration, increasing the predictor horizon monotonically improves out of
19 distribution generalization. The benchmark is released with reproducibility tooling,
20 a counterfactual replayer, and documented assumptions, revealing a critical insight
21 for world model design: out-of-distribution generalization requires temporally
22 structured bottlenecks that preserve long horizon dependencies, not merely longer
23 training contexts. Without ground truth access to the latent state, this dissociation
24 between context length and bottleneck structure remains invisible.

25 1 Introduction

26 World models [3, 2, 11, 12, 9] are increasingly proposed as general purpose representations for
27 complex dynamical systems, trained in a self-supervised manner to predict future state and spanning
28 joint embedding, recurrent latent dynamics, and autoregressive architectures. A central open question
29 is whether such representations capture the *physical* structure of the underlying system or merely
30 fit statistical regularities of the observation stream. Answering this requires paired access to sensor
31 observations *and* the true latent state, which existing physics and prognostics benchmarks [4, 15, 5, 16]
32 replace by a low dimensional readout (a plausibility verdict, a scalar remaining useful life) that cannot
33 certify whether a representation recovered the underlying causal structure.

34 We introduce **TurboSens**, an interactive turbofan simulator and accompanying dataset designed
35 as an *instrument* for evaluating world-model representations. TurboSens is generated using Open-
36 DeckSMR [13], a component-level thermodynamic aircraft engine simulator that exposes, at every

timestep, both a sensor stream and the full ten-dimensional ground-truth health state s_t . This paired access enables an *inverse evaluation protocol*: train a world model self-supervised on sensor sequences alone, freeze the encoder, then probe the latent space against the true s_t through regression, forecasting, and out-of-distribution tasks. Because the true state is available but never used as a training signal, TurboSens asks whether a learned representation recovers the causal structure of engine health from observations alone, going beyond scalar RUL regression and binary violation-of-expectation discrimination.

TurboSens is *interactive* rather than a static dump of trajectories. The simulator can be queried online under arbitrary action sequences, enabling counterfactual rollouts, reinforcement learning and model predictive control experiments, and on-demand generation of out-of-distribution regimes for stress testing. The environment is structured as an *open instrument*: degradation archetypes, maintenance policies, stochastic event rates, and flight phase contexts are independent and industrially grounded knobs, each anchored in a documented physical phenomenon. Scenarios are composed from the same building blocks and can be reconfigured to assemble progressively more challenging benchmarks as world model representations mature, rather than freezing a single reference difficulty level.

The simulator is driven by a well-maintained fleet abstraction: engines are kept within a healthy operating band by an expert policy combining life-limiting part time ceilings, health correlated scheduling, and opportunistic bundling, with imperfect restoration and deferred repairs. Trajectories therefore unfold not as a single monotonic arc but as a *sawtooth of partial resets*, punctuated by rare stochastic events (abstract step changes, transient pulses, sensor-level perturbations) that create discontinuities in the health state, the observations, or both. Sensor observations are recorded across multiple flight phase contexts per timestep, and the mapping from seven sensor channels to ten state dimensions is structurally underdetermined. Every modeling element is grounded in turbofan engine operating practice and is documented and configurable, so users can audit our assumptions independently of the protocol.

Contributions.

1. An **interactive turbofan simulator** built on OpenDeckSMR, exposing paired ground-truth state and sensor observations with discrete maintenance actions, on-demand trajectory generation, and a configurable cookbook of degradation archetypes, maintenance policies, events, and flight contexts.
2. An **inverse evaluation protocol** that trains world models self-supervised on sensor sequences, then directly probes their representations against the full ten-dimensional health state through regression, degradation-velocity estimation, multi-step latent forecasting, and out-of-distribution detection.
3. **Reference scenarios and splits**: released HDF5 datasets (TurboSens1, TurboSens2) with documented schemas, providing a concrete starting point for the protocol while leaving the pipeline open for further scenario composition.

2 Related Work

Why turbofan engines. We pick a turbofan engine as the substrate for world model evaluation for three reasons. First, the observations are scalar sensor channels rather than pixels, which removes the visual decoding bottleneck that dominates evaluation on video benchmarks and isolates the question of whether the encoder captures the underlying generative state. Second, the system carries a physically meaningful latent state (component level efficiency and flow capacity deviations) that the simulator emits at every timestep, so a probe against the ground truth state is well-posed and component-identifiable. Third, a single engine simultaneously exercises many phenomena that world models must handle in concert: noise, multi-context observations, action interventions through maintenance, region and phase-dependent dynamics, and rare stochastic events. The turbofan substrate also bridges world model evaluation with the predictive maintenance and digital twin literature, but the goal of this paper is squarely the former; the dataset is positioned as an instrument for evaluating world model representations, not as a prognostics benchmark.

World models and physical understanding. A growing family of world models, spanning joint-embedding predictive architectures [3, 2, 9], recurrent latent-dynamics models [11, 12], and

autoregressive sequence models [19, 8, 7, 6], train encoders and predictors on high-dimensional observation streams (pixels, sensors) to recover latent dynamics without supervision. A parallel body of work asks whether such representations encode *physical* structure: Physion [4] scores rigid-body outcome prediction, CLEVRER [20] probes causal reasoning over collision videos, and IntPhys/IntPhys2 [15, 5] adopt a violation-of-expectation paradigm. Two structural limitations recur across the field. First, these evaluations reduce physical understanding to binary discrimination, effectively an out-of-distribution detection task that flags anomalies rather than recovering the underlying state. Second, ground-truth access to the latent state is typically unavailable, so evaluation collapses onto downstream task return or reconstruction quality and cannot certify whether the representation captures physical structure or merely statistical regularities of the observation stream.

Interactive simulators and agentic environments. Gym-style interactive environments are the standard testbed for reinforcement learning and model predictive control [18, 17], but industrial degradation settings have received comparatively little attention in this form. Maulana et al. [10] argue for maintenance policies that reason over longer action horizons, a setting that requires an online simulator rather than a static dataset. No existing turbofan benchmark exposes this interface: an agent cannot apply a maintenance action and observe the consequence.

Turbofan degradation datasets. Turbofan prognostics research has long been dominated by C-MAPSS [16] and N-CMAPSS [1], which pair sensor streams with a scalar remaining-useful-life label as the sole evaluation target [14]. These datasets established the field but expose neither the full latent health state nor an interactive interface.

3 Problem Formulation

Evaluating a world model on TurboSens means asking whether its representation captures the underlying generative structure of the data: the latent state, its temporal dynamics, the action effects that intervene on it, and the noise that obscures it. We build the formulation up in three steps, starting from the simplest version of the simulator and adding components only as we need them 1.

Step 1 observation, state, and the OpenDeckSMR map. At every flight cycle t the engine has a latent mechanical wear state $\mathbf{s}_t \in \mathbb{R}^K$ with $K=10$, encoding efficiency and flow capacity deviations of the five sub systems (booster, fan, high pressure compressor, high pressure turbine, low pressure turbine). The thermodynamic simulator OpenDeckSMR then produces a sensor observation tensor

$$\mathbf{o}_t = g_{\text{ODS}}(\mathbf{s}_t, \phi_t) + \boldsymbol{\varepsilon}_t \in \mathbb{R}^{S \times C}, \quad (1)$$

with $S=7$ sensor channels measured under C flight phase contexts ($C=12$ in TurboSens1, $C=16$ in TurboSens2), ϕ_t the current context, and $\boldsymbol{\varepsilon}_t$ sensor noise. The probing target throughout the paper is $\mathbf{s}_t$; everything else either drives its dynamics or shapes how it is observed.

Step 2 TurboSens1: degradation archetypes, weather, maintenance. TurboSens1 augments $(\mathbf{s}_t, \mathbf{o}_t)$ with three components that meaningfully shape state trajectories. Each episode is assigned a *degradation archetype* $\eta \in \{A, B, C, D\}$ that acts as a per sub system drift multiplier (compressor heavy, fan and booster heavy, turbine heavy, balanced) and produces qualitatively different wear curves across the fleet. A *weather* channel w_t (ambient temperature deviation, seasonal phase) modulates the operating envelope and the per step drift. A discrete *maintenance action* $a_t \in \mathcal{A}_3$ ($|\mathcal{A}_3|=6$) restores selected components of $\mathbf{s}_t$ via a diagonal restoration mask $R(a_t) \in \{0, 1\}^{K \times K}$ with imperfect efficacy $\boldsymbol{\rho}_{\text{mech}}$. The state then evolves

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \boldsymbol{\Delta}_s(\mathbf{s}_t, \eta, w_t) + R(a_t) \boldsymbol{\rho}_{\text{mech}}(a_t, \eta) + \boldsymbol{\eta}_t, \quad (2)$$

where $\boldsymbol{\Delta}_s$ collects the archetype and weather conditional drift and $\boldsymbol{\eta}_t$ is zero mean process noise. Stochastic events (abstract permanent step changes, transient anomalies, sensor level perturbations) inject sparse, structured discontinuities on top of equation 2; their event flag vector $\mathbf{e}_t$ is part of the latent state but does not enter the probing target. Although the events draw inspiration from real engine phenomena, their probabilities and magnitudes are mathematical abstractions and in no case reflect what is observed in real life flight settings; calibrating them to operational data is left for future work.

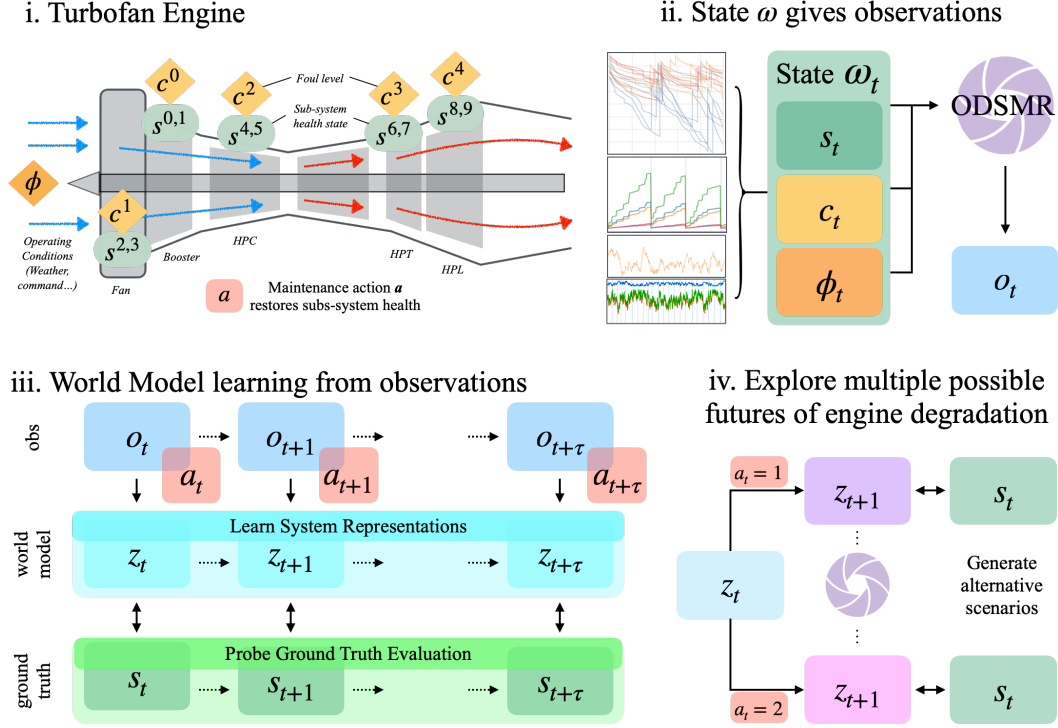


Figure 1: **The TurboSens evaluation framework.** (i) A turbofan engine is decomposed into subsystems whose health degrades over time, modulated by operating context, fouling, and maintenance actions a that restore selected components of the health state. (ii) Subsystem health and fouling channels assemble into a global state ω that OpenDeckSMR (ODSMR) maps to sensor observations across C flight phase contexts. (iii) A world model learns a representation from these observations in a self supervised manner; at test time, the frozen representations are probed against the true subsystem health, framing representation evaluation as an inverse problem. (iv) Companion downstream tasks include forecasting possible futures conditioned on candidate maintenance policies.

136 **Step 3 TurboSens2: fouling, phantom observation, regions, wash actions.** TurboSens2 keeps
 137 the state equation 2 for mechanical wear and adds a transient *fouling* channel $\mathbf{c}_t \in \mathbb{R}_{\geq 0}^{K_c}$ ($K_c=5$, one
 138 per module) that fouls module efficiency and biases corrected flow upward. The action set extends to
 139 $\mathcal{A}_4$ ($|\mathcal{A}_4|=7$) with an *engine wash* action that decays $\mathbf{c}_t$ toward zero without touching $\mathbf{s}_t$:

$$\mathbf{c}_{t+1} = \gamma(a_t) \mathbf{c}_t + \mathbf{u}_t(\phi_t, \eta_{\text{reg}}) + \zeta_t, \quad (3)$$

140 with $\gamma(a_t)$ the decay factor (close to one for non cleaning actions, small for a wash) and $\mathbf{u}_t$ a region
 141 and phase dependent fouling source. Episodes are also drawn from one of six geographic regions
 142 η_{reg} that shift event rates, archetype priors, and fouling dynamics.

143 The two channels do not reach the sensors independently. They are first combined into a *phantom*
 144 *state*

$$\tilde{\mathbf{s}}_t = \text{clip}(\mathbf{s}_t + M \max(\mathbf{0}, -\mathbf{s}_t) + C \mathbf{c}_t), \quad (4)$$

145 where M is a sparse non negative coupling that lets local wear inflate downstream flow capacity and
 146 C is a sparse non negative coupling that lets fouling inflate corrected flow and depress efficiency. The
 147 simulator is then queried on $\tilde{\mathbf{s}}_t$ rather than $\mathbf{s}_t$:

$$\mathbf{o}_t = g_{\text{ODS}}(\tilde{\mathbf{s}}_t, \phi_t) + \varepsilon_t. \quad (5)$$

148 A single $\mathbf{o}_t$ is therefore consistent with many $(\mathbf{s}_t, \mathbf{c}_t)$ pairs, and the observability of $\mathbf{s}_t$ is action
 149 dependent: immediately after a wash, $\mathbf{c}_t \approx 0$ so $\tilde{\mathbf{s}}_t \approx \mathbf{s}_t$ and the sensors expose the true wear;
 150 between washes fouling rebuilds and the two diverge. A representation that simply regresses $\tilde{\mathbf{s}}_t$ from
 151 $\mathbf{o}_t$ is therefore mis specified, and only a world model that integrates the action history can disentangle
 152 the two channels.

POMDP form and evaluation task. Collecting $(\mathbf{s}_t, \mathbf{c}_t, w_t, \phi_t, \eta_{\text{reg}}, e_t)$ into a single latent $\omega_t \in \Omega$ recovers the standard partially observable Markov decision process $(\Omega, \mathcal{A}, \mathcal{O}, f, g, \rho_0)$, which we use in Appendix A for the channel wise transitions in their general form. We ask whether a world model trained self supervised on observation and action histories learns representations that recover the engine state $\mathbf{s}_t$. The encoder produces an embedding

$$\mathbf{z}_t = \text{Enc}_\theta(\mathbf{o}_{\leq t}, a_{\leq t}) \in \mathbb{R}^d, \quad (6)$$

which we freeze and feed to a fixed linear probe h_ψ trained to predict

$$\hat{\mathbf{s}}_{t+\tau} = h_\psi(\mathbf{z}_t) \quad \text{for } \tau \geq 0, \quad (7)$$

reporting per component Pearson, RMSE, and R^2 on held out episodes (Section 5).

4 The TurboSens Dataset

Symbol	Description
$\omega_t \in \Omega$	Full POMDP latent state (see eq. ??)
$\mathbf{s}_t \in \mathbb{R}^K$	Engine mechanical wear, probing target
$\mathbf{c}_t \in \mathbb{R}_{\geq 0}^{K_c}$	Fouling channel (TurboSens2 only)
$\tilde{\mathbf{s}}_t$	Phantom state coupling $\mathbf{s}_t$ and $\mathbf{c}_t$
K, K_c	Mechanical and fouling dimensions ($K=10$)
ϕ_t	Flight phase context
S, C	Sensor types and flight phase contexts
$\mathbf{o}_t \in \mathbb{R}^{S \times C}$	Sensor observation tensor at time t
$a_t \in \mathcal{A}$	Maintenance action at time t
$\mathbf{z}_t \in \mathbb{R}^d$	Encoder embedding, $\text{Enc}_\theta(\mathbf{o}_{\leq t}, a_{\leq t})$
$\hat{\mathbf{s}}_{t+\tau}$	Probe prediction at horizon τ

Table 1: Core notation used in the main body; the full symbol table (probe, objective, and scenario specific symbols) is in Appendix B.

Property	TurboSens1	TurboSens2
Episodes	400	500 (train) / 60 (test) / 60 (test-hard)
Total flights	4M	7.0M (train) / 831k (test) / 855k (test-hard)
Episode length (mean/max)	10,000 / 10,000	14,022 / 20,000
Observation $\mathbf{o}_t$	$\mathbb{R}^{7 \times 12}$	$\mathbb{R}^{7 \times 16}$
Observation dim $S \cdot C$	84	112
Mechanical wear dim K	10	10
Fouling dim K_c	—	5
Maintenance policy	health based	health based + context
Action space $\mathcal{A}$	discrete (6)	discrete (7, adds engine wash)
Repair rate (per step)	0.04%	0.09%
Wash rate (per step)	—	1.05%
Regions	0	6
Degradation archetypes	4	4
Stochastic events	10	12 (adds two regional events)
Phantom observation model	no	yes, $\tilde{\mathbf{s}}_t \neq \mathbf{s}_t$
Weather model	limited	hierarchical seasonal OU
Flight-phase contexts C	12	16
Context meaning	key probes	flight-phase discretization

Table 2: OpenDeckSMR environment properties across scenarios. Episode length for TurboSens2 is the mean across the train split. The repair rate counts steps with any non-wash maintenance action; the wash rate counts engine-wash actions separately since TurboSens1 has no fouling channel to clean.

TurboSens is a synthetic turbofan degradation dataset generated with OpenDeckSMR, a component level thermodynamic simulator that exposes *both* the observable sensor stream and the ground truth health state $\mathbf{s}_t$, the key property that enables direct evaluation of learned representations. At every

flight cycle, the simulator is queried at C fixed flight phase contexts (climb, cruise, descent, etc.; $C=12$ in TurboSens1, $C=16$ in TurboSens2) and emits an observation tensor $\mathbf{o}_t \in \mathbb{R}^{S \times C}$ with $S=7$ sensor channels per context. The multi context structure is deliberate: different flight conditions excite different parts of the thermodynamic cycle, so the C slices of $\mathbf{o}_t$ together carry more information about $\mathbf{s}_t$ than any single context in isolation, and a world model that learns to aggregate evidence across contexts is rewarded.

We release two scenarios, summarized in Table 2. **TurboSens1** offers health-correlated maintenance with six actions, four heterogeneous degradation archetypes, twelve flight contexts, a library of ten stochastic events, and imperfect restoration. Full details of Scenario 1 are provided in Appendix C.1. **TurboSens2** extends this setting with a two-channel hidden state that separates intrinsic mechanical wear from transient fouling, six geographic operating regions with region-specific event intensities, sixteen flight-phase contexts, and a phantom observation model in which the sensed state $\tilde{\mathbf{s}}$ can diverge from the true mechanical state $\mathbf{s}$ under heavy fouling. The transition from TurboSens1 to TurboSens2 introduces phantom observations, a fouling channel, an engine wash action, and region conditional weather and event dynamics; the difference between probe results on the two scenarios is therefore an end to end ablation of these features taken jointly. Per feature attribution is left for future work.

TurboSens is designed to challenge world models with sparse, imbalanced interventions (component overhauls occur on 0.04%-0.09% of steps, engine washes add another $\approx 1\%$ in TurboSens2), heterogeneous failure trajectories, multi context partial observability, and the action dependent latent observability of the phantom map (eq. 19). The dataset is generated on demand via a provided script; the reference splits are released as HDF5 files with a documented schema.

4.1 Datasets Coverage

4.2 Degradation and Maintenance Mechanisms

TurboSens1 models degradation as a two-regime process: an early *burn-in* phase superimposed on a persistent linear drift. For component k , the per-step increment is

$$\Delta s_t^k = -\beta_k \exp(-t_k/\tau_k) - \alpha_k^{(m)} + \varepsilon_t, \quad (8)$$

where t_k is the time since last repair, β_k, τ_k control burn-in, $\alpha_k^{(m)}$ is a speed-regime-dependent drift rate, and ε_t is Gaussian noise. The linear term dominates after ≈ 500 steps, consuming ~ 70 -80% of the degradation budget over a 10^4 -step episode.

Heterogeneous archetypes. Each episode is assigned one of four failure modes (Table 7), modulating drift rates to concentrate wear in specific subsystems. Archetypes A-C mimic real-world degradation; Archetype D (balanced) produces long-lived engines that rarely reach airworthiness limits.

Table 3: Degradation archetypes in TurboSens1.

ID	Name	Drift multiplier	Dominant subsystem
A	Compressor-driven	up to $2.8\times$	HPC, Booster
B	Fan/Booster-driven	up to $2.5\times$	Fan, Booster
C	Turbine-driven	up to $2.8\times$	HPT, LPT
D	Balanced	$0.65\times$	All uniformly

Realistic maintenance. Six actions (idle or replace one of five module assemblies) are selected by an expert policy that mimics industrial practice:

- *LLP ceilings*: hard cycle limits per component, forcing replacement regardless of health.
- *Health-correlated scheduling*: maintenance is triggered when a component is predicted to reach 80% degradation within 1500-2500 steps.
- *Opportunistic bundling*: any visit triggers a scan for other near-end-of-life components, bundling replacements to avoid costly separate shop visits.

- *Imperfect restoration*: even new components restore only 80-90% of the degradation budget, with episode-varying quality $\kappa \sim \mathcal{U}(0.7, 1.0)$.
- *Inspection deferred repairs*: 3-7% chance that a degraded component is missed during inspection, forcing an unplanned emergency visit 200-800 steps later.

This policy yields sparse interventions (0.04% of timesteps) and strongly asymmetric inter-maintenance intervals, in contrast to the uniform scheduling of earlier benchmarks.

A library of ten stochastic events (`perm_step_1`, `trans_anomaly_1`, `sensor_pulse_1`, and others) further enriches the dataset, making the latent state recovery problem harder and the encoder representations more challenging to learn. Although these events draw inspiration from real engine phenomena, their probabilities and magnitudes are mathematical abstractions and in no case reflect what is observed in real life flight settings; calibrating them to operational data is left for future work. Full details, rates, and the mapping between labels and the mathematical effects they model are provided in Appendix C.1.4.

5 Evaluation Framework

218

The benchmark proposes an inverse evaluation protocol. A world model is trained self supervised on the sensor observation stream of the train split; we then freeze the encoder and probe its embeddings against the ground truth mechanical state $\mathbf{s}_t$ that the simulator exposes. The protocol is architecture agnostic: any encoder that produces a per frame embedding $\mathbf{z}_t$ admits the same probe without modification, and no fine tuning of the world model is performed. Two evaluation tasks are central to the present paper; two auxiliary tasks (degradation velocity estimation and latent forecasting with an action divergence gap) ship with the benchmark and are described in Appendix F.

Task 1 Health Indicator state estimation. A linear probe h_ψ is trained to predict $\mathbf{s}_t \in \mathbb{R}^{10}$ from the embedding $\mathbf{z}_t$ on the train split’s full $\sim 770\text{k}$ step embedding stream. The probe is evaluated on the in distribution test split and the OOD test_hard split (Section 5). We report mean and per dimension Pearson, RMSE, and R^2 across the ten components, with multi seed (mean $\pm$ std) error bars where seeds are available. Probe head capacity is held fixed across runs after observing that the probe head matters far less than the probe data budget (Appendix E.5).

Task 2 OOD probing. TurboSens2 ships an OOD test_hard split (60 episodes, distinct seed offset, 6.7% extreme degradation episodes versus 5.0% in test) drawn from the same generator and region prior; the marginals of $\mathbf{s}_t$ in test_hard concentrate mass at more degraded values, exposing the encoder to state regions less represented in the train split. We report Task 1 metrics on both splits and the in distribution / OOD gap as the generalisation measure. The companion latent surprise score $\mathcal{S}(t) = \|\mathbf{z}_{t+1} - g_\psi(\mathbf{z}_{t-H+1:t}, \mathbf{a}_{t-H+1:t})\|_2$ is well defined for any model exposing a one step latent prediction and is reported alongside the supervised probes (Appendix F).

Probe over single embedding versus full history. The probe head supports both a single embedding input $h_\psi(\mathbf{z}_t)$ (`seq_len=1`) and a temporally aggregated input $h_\psi(\mathbf{z}_{t-w+1:t})$ over a window of w consecutive embeddings. The default reported in Section 5.1 is `seq_len=1` since it isolates how much of $\mathbf{s}_t$ a single embedding carries; an `seq_len=10` comparison is reported alongside in the released eval pipeline and in Appendix E.4.

5.1 Results

We sweep two architectures over the temporal context axis on TurboSens2. Both share the SensorEncoder of Appendix D.1 and differ only in the predictor: a JEPA transformer predictor (828k parameters) and an autoregressive LSTM predictor (1.14M parameters), both trained 10 epochs self supervised on the train split with AdamW + cosine schedule and bf16 mixed precision. We sweep predictor history $H \in \{8, 16, 32\}$ and stride $S \in \{1, 5, 10, 20\}$ at fixed horizon $P=4$ and effective batch 512. JEPA runs report multi seed mean $\pm$ std over up to six independent seeds (one degenerate seed per cell is filtered); AR LSTM runs L_1 , L_2 , L_{big} are single seed and reported as preliminary. Wall clock and configuration details are in Appendix E.1.

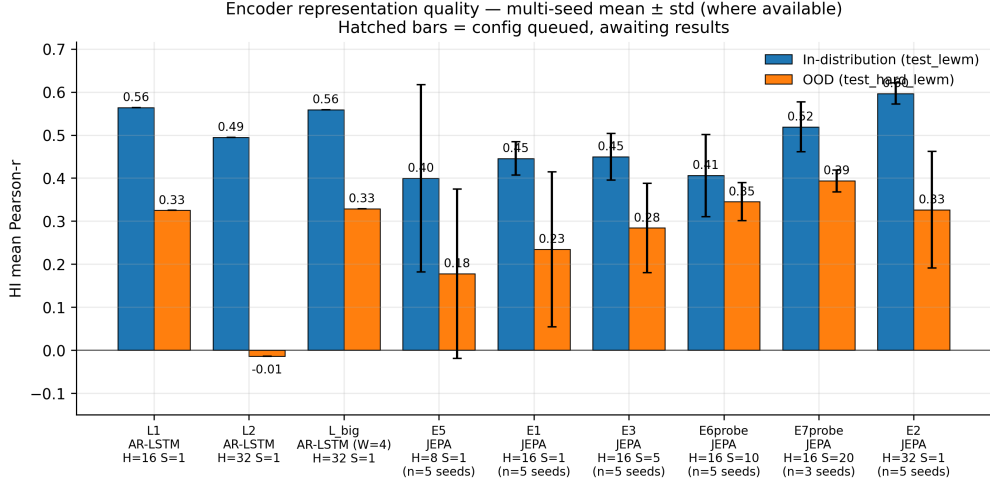


Figure 2: Encoder representation quality across two architectures and seven temporal configurations on TurboSens2. Mean HI Pearson on the in distribution test split (blue) and the OOD test_hard split (orange); JEPA error bars are multi seed std ($n=5$ to 6 valid seeds, one degenerate seed filtered); AR LSTM rows (L_1 , L_2 , L_{big}) are single seed. Inside JEPA the in distribution leader is E_2 and the OOD leader is E_7 at 0.394 ± 0.025 , with the OOD trend monotone in stride at $H=16$. The scatter view in Appendix E.7 places each configuration relative to the identity line.

253 **Stride S monotonically improves OOD generalisation.** Figure 2 reports mean HI Pearson on
 254 the in distribution test split and the OOD test_hard split. Inside the JEPA family at $H=16$,
 255 OOD Pearson is monotonic in stride: $S=1$ (E_1 , 0.234 ± 0.180) $\rightarrow$ $S=5$ (E_3 , 0.284 ± 0.104) $\rightarrow$
 256 $S=10$ (E_6 , 0.345 ± 0.044) $\rightarrow$ $S=20$ (E_7 , **0.394 ± 0.025** , the tightest std of any cell). E_7 is also
 257 the second strongest configuration in distribution at 0.519 ± 0.058 , behind only E_2 ($H=32$, $S=1$,
 258 **0.597 ± 0.025**). Stretching the stride therefore gives up only a small fraction of in distribution
 259 Pearson and recovers most of the OOD gap; shrinking the history (E_5 , $H=8$) collapses on OOD
 260 (-0.03 ± 0.09). Multi seed averaging is essential: a single seed of E_1 would have placed it on top at
 261 0.56 , off the multi seed mean by roughly one standard deviation.

262 **Per-component behaviour: Pearson and RMSE disagree on rankings.** The per-component
 263 breakdown of the in distribution leader E_2 (Appendix E.3, Figure 10) sign flips on three of ten
 264 dimensions on OOD and exhibits Pearson and RMSE rankings that disagree: HI_2 has an acceptable
 265 OOD Pearson but the worst RMSE, while HI_8 has a high in distribution Pearson and a moderate
 266 one on OOD. Single metric reporting hides this; we recommend per component Pearson and RMSE
 267 alongside the mean and treat the worst component as a more conservative figure of merit under shift.

268 **Baseline anchors.** Two baselines anchor the trained encoder numbers. A raw_sensors probe runs
 269 the identical eval pipeline but feeds the linear head the raw 176 dimensional observation tensor instead
 270 of the encoder embedding; it is a diagnostic of how much of s_t is linearly recoverable from the raw
 271 observation under each split. A random_encoder baseline keeps the encoder architecture untrained
 272 and otherwise matches the trained pipeline. The supervised_oracle trains the same SensorEncoder
 273 backbone end to end with a linear head under direct MSE supervision on s_t , then is evaluated by the
 274 same probe pipeline; it is the practical ceiling of what the encoder backbone can preserve. Table 4
 275 stacks the trained checkpoints alongside both diagnostics. Both the oracle and raw_sensors have a
 276 near zero in distribution / OOD gap ($0.545 \rightarrow 0.543$ and $0.509 \rightarrow 0.506$), while every self supervised
 277 encoder we trained has a substantial OOD gap. The OOD relevant signal is therefore in the data and
 278 recoverable by either ceiling; the gap is the headline quantity the protocol exposes.

279 **Counterfactual fidelity.** The deterministic EpisodeReplayer (Appendix F.2) allows comparing
 280 each trained predictor against the simulator under counterfactual action sequences. Across five
 281 checkpoints $\times$ seven actions $\times$ thirty episodes $\times$ three branch positions, the absolute HI RMSE
 282 between model and sim plateaus at ≈ 0.003 – 0.005 within $\tau \approx 20$ and remains bounded through

Table 4: Mean HI Pearson on test and OOD test_hard (probe seq_len=1). Seeds: valid seeds out of five or six (degenerate probe init seeds filtered). The supervised oracle and raw_sensors are diagnostic baselines (ceiling and floor); both have a near zero OOD gap.

encoder	test Pearson	test_hard Pearson	seeds
Supervised oracle (ceiling)	0.545 ± 0.229	0.543 ± 0.046	4/5
raw_sensors (no encoder)	0.509 ± 0.386	0.506 ± 0.043	3–4/5
E_2 JEPA, $H=32, S=1$	0.597 ± 0.025	0.327 ± 0.135	5/6
E_7 JEPA, $H=16, S=20$	0.519 ± 0.058	0.394 ± 0.025	3/3
E_6 JEPA, $H=16, S=10$	0.406 ± 0.095	0.345 ± 0.044	5/6
E_3 JEPA, $H=16, S=5$	0.450 ± 0.054	0.284 ± 0.104	5/5
E_1 JEPA, $H=16, S=1$	0.446 ± 0.039	0.234 ± 0.180	5/5
E_5 JEPA, $H=8, S=1$	0.400 ± 0.218	0.178 ± 0.196	5/5
L_{big} AR LSTM, $H=32, S=1, W=4$	0.511 ± 0.067	0.344 ± 0.048	5/5
L_2 AR LSTM, $H=32, S=1$	0.522 ± 0.041	0.231 ± 0.096	4/5
L_1 AR LSTM, $H=16, S=1$	0.554 ± 0.015	0.117 ± 0.142	4/5

283 $\tau=100$; the differential $|\Delta_{\text{model}}(a, 0) - \Delta_{\text{sim}}(a, 0)|$ that isolates the action specific response sits
284 at $\approx 0.0015\text{--}0.005$, non zero across all configurations. The (H, S) axis that strongly differentiates
285 probe Pearson barely differentiates counterfactual fidelity, suggesting the within JEPA specificity vs
286 generality tradeoff lives at the encoder rather than the predictor (per action and per checkpoint curves
287 in Appendix E.6, Figure 11).

288 **More temporal context closes the OOD gap, across architectures.** The multi seed AR LSTM
289 rows reveal a monotone OOD trend in temporal context: L_1 (OOD 0.117) $\rightarrow L_2$ (OOD 0.231) $\rightarrow$
290 L_{big} (OOD 0.344), with both predictor history and a TemporalAggregator window contributing.
291 Combined with the JEPA stride trend ($S=1 \rightarrow 20$: OOD 0.234 $\rightarrow$ 0.394 at $H=16$), the picture
292 is consistent across architectures: more temporal context during pretraining reliably shrinks the in
293 distribution / OOD gap, regardless of how that context enters (stride for JEPA, history and aggregator
294 width for AR LSTM).

295 **Action conditioned counterfactuals (ongoing).** The released simulator exposes a determinis-
296 tic EpisodeReplayer (Appendix F.2) that reconstructs any episode bit identically from its par-
297 ent seed and forks it at a chosen branch point under either an overridden action (branch_t +
298 override_action), an open loop forced action sequence (forced_actions), or a re seeded
299 weather realisation (weather_seed). This is the python interface around which the action con-
300 ditioned probe will be reported in the camera ready: rolling the trained predictor under the same
301 counterfactual action sequence as the replayer and measuring the divergence between predicted and
302 replayed mechanical state closes the last cell of the evaluation framework of Section 5.

303 5.2 Discussion

304 The multi seed sweep makes three deferred repairs visible. (i) Increasing the temporal context during
305 pretraining monotonically improves OOD generalisation across architectures (JEPA stride at $H=16$:
306 OOD 0.234 $\rightarrow$ 0.394; AR LSTM history $\times$ aggregator width: OOD 0.117 $\rightarrow$ 0.344). (ii) Two
307 diagnostics bracket the SSL encoders and reframe the OOD picture: the supervised oracle and a no
308 encoder raw probe both reach $\approx 0.55/0.51$ on test / test_hard with essentially zero gap, while
309 even the strongest SSL encoder sits ≈ 0.15 Pearson below the supervised ceiling on the held out split.
310 The encoder backbone can preserve the OOD relevant information when supervised and the signal
311 is present in the observation; the gap is therefore a property of the SSL pretext, not the backbone
312 or the data. Closing this gap is the research question the benchmark surfaces. (iii) Counterfactual
313 fidelity is bounded across all checkpoints and barely differentiated by (H, S) , locating the within
314 JEPA specificity vs generality tradeoff at the encoder level rather than the predictor level.

315 Two methodological points underpin all three deferred repairs. Single seed probe numbers can be off
316 by more than one standard deviation: the multi seed reranking flipped the JEPA in distribution leader
317 from E_1 to E_2 . And aggregate metrics hide a per component picture in which Pearson and RMSE
318 rank components differently and sign flips appear on multiple dimensions for the in distribution

319 leader E_2 ; the worst component is therefore a more conservative summary statistic than the mean
 320 under shift.

321 6 Conclusion

322 TurboSens is an evaluation instrument for world model representations. Its distinctive property of
 323 paired ground truth latent state alongside the sensor stream, supports concrete evaluative claims: how
 324 much of the underlying state a frozen representation preserves under distribution shift, whether gener-
 325 alisation gaps trace to the encoder or the predictor, and how the predictor’s response to counterfactual
 326 actions compares to the simulator’s. The contribution is the instrument itself: a configurable simulator,
 327 two progressively challenging scenarios, a linear probing protocol, a deterministic counterfactual
 328 replayer, and multi seed reproducibility tooling.

329 Our experiments illustrate the instrument. On TurboSens2, more temporal context during pretraining
 330 monotonically improves OOD probe quality across both architectures we tested, and two diagnostics
 331 (a supervised oracle on the same backbone, and a linear probe on the raw observation) both close
 332 the in distribution / OOD gap entirely while every SSL encoder leaves ≈ 0.15 Pearson on the table.
 333 The OOD relevant signal is therefore in the data and the encoder backbone can preserve it under
 334 supervision; the gap is a property of the SSL pretext. The benchmark documents its assumptions and
 335 limitations (synthetic data, single domain, simulator fidelity, linear probe sufficiency), and progressive
 336 scenarios let researchers isolate specific evaluation challenges while keeping ground truth accessible.

337 References

338 References

- 339 [1] Manuel Arias Chao, Chetan Kulkarni, Kai Goebel, and Olga Fink. Aircraft engine run-to-failure
 340 dataset under real flight conditions for prognostics and diagnostics. *Data*, 6(1):5, January 2021.
 341 ISSN 2306-5729. doi: 10.3390/data6010005. URL [https://www.mdpi.com/2306-5729/](https://www.mdpi.com/2306-5729/6/1/5)
 342 [6/1/5](https://www.mdpi.com/2306-5729/6/1/5).
- 343 [2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba, Komeili,
 344 Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud,
 345 Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil
 346 Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li,
 347 Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas
 348 Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning.
 349 (arXiv:2506.09985), 2025. doi: 10.48550/arXiv.2506.09985. URL [http://arxiv.org/abs/](http://arxiv.org/abs/2506.09985)
 350 [2506.09985](http://arxiv.org/abs/2506.09985). arXiv:2506.09985 [cs].
- 351 [3] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning
 352 without the heuristics. (arXiv:2511.08544), November 2025. doi: 10.48550/arXiv.2511.08544.
 353 URL <http://arxiv.org/abs/2511.08544>. arXiv:2511.08544 [cs].
- 354 [4] Daniel Bear, Elias Wang, Damian Mrowca, Felix Jedidja Binder, Hsiao-Yu Tung, RT Pramod,
 355 Cameron Holdaway, Sirui Tao, Kevin A Smith, Fan-Yun Sun, et al. Physion: Evaluating
 356 physical prediction from vision in humans and machines. In *Thirty-fifth Conference on Neural*
 357 *Information Processing Systems Datasets and Benchmarks Track (Round 1)*.
- 358 [5] Florian Bordes, Quentin Garrido, Justine T. Kao, Adina Williams, Michael Rabbat, and Em-
 359 manuel Dupoux. Intphys 2: Benchmarking intuitive physics understanding in complex syn-
 360 thetic environments. (arXiv:2506.09849), 2025. doi: 10.48550/arXiv.2506.09849. URL
 361 <http://arxiv.org/abs/2506.09849>. arXiv:2506.09849 [cs].
- 362 [6] Francesca Calabrese, Alberto Regattieri, Marco Bortolini, Mauro Gamberi, and Francesco Pilati.
 363 Predictive maintenance: A novel framework for a data-driven, semi-supervised, and partially
 364 online prognostic health management application in industries. *Applied Sciences*, 11(8):3380,
 365 2021.

- [7] Weikun Deng, Khanh TP Nguyen, Christian Gogu, Kamal Medjaher, and Jérôme Morio. Enhancing prognostics for sparse labeled data using advanced contrastive self-supervised learning with downstream integration. *Engineering Applications of Artificial Intelligence*, 138: 109268, 2024.
- [8] Rayan Krishnan, Pranav Rajpurkar, and Eric J Topol. Self-supervised learning in medicine and healthcare. *Nature Biomedical Engineering*, 6(12):1346–1352, 2022.
- [9] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. (arXiv:2603.19312), March 2026. doi: 10.48550/arXiv.2603.19312. URL <http://arxiv.org/abs/2603.19312>. arXiv:2603.19312 [cs].
- [10] Faisal Maulana, Andrew Starr, and Agusmian Partogi Ompusunggu. Explainable data-driven method combined with bayesian filtering for remaining useful lifetime prediction of aircraft engines using nasa cmapss datasets. *Machines*, 11(2):163, January 2023. ISSN 2075-1702. doi: 10.3390/machines11020163. URL <https://www.mdpi.com/2075-1702/11/2/163>.
- [11] Masashi Okada and Tadahiro Taniguchi. Dreaming: Model-based reinforcement learning by latent imagination without reconstruction. In *2021 IEEE international conference on robotics and automation (icra)*, pages 4209–4215. IEEE, 2021.
- [12] Masashi Okada and Tadahiro Taniguchi. Dreamingv2: Reinforcement learning with discrete world models without reconstruction. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 985–991. IEEE, 2022.
- [13] Michail Psaropoulos, Vasilis Gkoutzamanis, Anestis I. Kalfas, Panagiotis Giannakakis, Sebastien Razakarivony, Solene Thepaut, and Don Quan Vu. Opendecksmr, December 2025. URL <https://doi.org/10.5281/zenodo.17804895>.
- [14] Emmanuel Ramasso and Abhinav Saxena. *Review and Analysis of Algorithmic Approaches Developed for Prognostics on CMAPSS Dataset*. 2015. URL <https://ntrs.nasa.gov/api/citations/20150007677/downloads/20150007677.pdf>.
- [15] Ronan Riochet, Mario Ynocente Castro, Mathieu Bernard, Adam Lerer, Rob Fergus, Véronique Izard, and Emmanuel Dupoux. Intphys: A framework and benchmark for visual intuitive physics reasoning. *arXiv preprint arXiv:1803.07616*, 2018.
- [16] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 International Conference on Prognostics and Health Management*, page 1–9, Denver, CO, USA, October 2008. IEEE. ISBN 978-1-4244-1935-7. doi: 10.1109/PHM.2008.4711414. URL <http://ieeexplore.ieee.org/document/4711414/>.
- [17] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.
- [18] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- [19] Tian Wang, Meina Qiao, Mengyi Zhang, Yi Yang, and Hichem Snoussi. Data-driven prognostic method based on self-supervised learning approaches for fault detection. *Journal of Intelligent Manufacturing*, 31(7):1611–1619, 2020.
- [20] Kexin Yi, Chuang Gan, Yunzhu Li, Pushmeet Kohli, Jiajun Wu, Antonio Torralba, and Joshua B Tenenbaum. Clevrer: Collision events for video representation and reasoning. In *International Conference on Learning Representations*.

413	Appendix Contents	
414	A POMDP and Observation Model	13
415	B Full Notation	14
416	C Description of the Environments	14
417	C.1 The Scenario 1 Turbofan Degradation Dataset	15
418	C.2 TurboSens2: Regional Operating Environments and Fouling Dynamics	19
419	D Architecture and Implementation Details	27
420	D.1 Sensor Encoder	27
421	D.2 Temporal Aggregator	28
422	D.3 World Model and Loss	28
423	D.4 Predictor and AR LSTM Baseline	29
424	D.5 Training Setup	29
425	D.6 Downstream Evaluation Probes	30
426	E TurboSens2 Sweep Campaign Details	31
427	E.1 Run inventory	31
428	E.2 VRAM scaling and effective sequence length	32
429	E.3 Per HI breakdown of E_2 and E_1	32
430	E.4 Probe over single embedding versus full history	33
431	E.5 Probe data size sensitivity	33
432	E.6 Counterfactual fidelity curves	33
433	E.7 Cross split scatter	34
434	F Auxiliary Evaluation Tasks	35
435	F.1 Degradation velocity estimation	35
436	F.2 Deterministic replay and counterfactual API	35
437	F.3 Latent forecasting with action divergence gap	35
438	G Illustrative Trajectories and Dataset Views	35

439 A POMDP and Observation Model

440 **Latent state.** An episode of TurboSens is a trajectory of the partially observable Markov decision
441 process

$$\mathcal{M} = (\Omega, \mathcal{A}, \mathcal{O}, f, g, \rho_0),$$

442 with latent state

$$\omega_t = (\mathbf{s}_t, \mathbf{c}_t, w_t, \phi_t, \eta_{\text{reg}}, e_t) \in \Omega,$$

443 where $\mathbf{s}_t \in \mathbb{R}^K$ is the persistent mechanical wear ($K=10$), $\mathbf{c}_t \in \mathbb{R}_{\geq 0}^{K_c}$ is the transient fouling channel
444 (zero in TurboSens1), w_t is the weather state (ambient temperature deviation, precipitation gate),
445 $\phi_t \in \mathcal{C}$ is the current flight phase context with $|\mathcal{C}|=C$, η_{reg} is the episode constant region and

archetype label, and $e_t \in \{0, 1\}^{n_{\text{evt}}}$ is the current stochastic event flag vector. The engine mechanical state $\mathbf{s}_t$ is the probing target; the other channels are carried because they shape its dynamics or the observation map.

Transitions. The channels of ω_t evolve under a deterministic conditional drift f (conditional on the region and archetype label η_{reg} and the current weather and phase) which is then perturbed by process noise ξ_t and modulated by the maintenance action a_t . The action enters multiplicatively through a per channel factor $\Lambda_k(a_t)$ and additively through a restoration source $\rho_k(a_t, \omega_t)$. The transition is therefore deterministic in f but stochastic overall through ξ_t and the discrete event mask e_t :

$$\omega_{t+1}^{(k)} = \Lambda_k(a_t) f_k(\omega_t) + \rho_k(a_t, \omega_t) + \xi_t^{(k)}, \quad k = 1, \dots, n_{\text{ch}}. \quad (9)$$

When several independent modulations act on the same channel (for instance the fouling decay factor of a cleaning action combined with a context dependent deposition gate), the per channel factor is itself a product series

$$\Lambda_k(a_t) = \prod_{j=1}^{J_k} \lambda_{k,j}(a_t), \quad (10)$$

so the joint effect of the action across channels is captured by the product $\prod_{k,j} \lambda_{k,j}(a_t)$ rather than by a single transition matrix. Concretely, for the two channels that carry most of the paper’s signal,

$$\mathbf{s}_{t+1} = \mathbf{s}_t + \Delta_s(\mathbf{s}_t, \eta_{\text{reg}}) + R(a_t) \rho_{\text{mech}}(a_t, \eta_{\text{reg}}) + \boldsymbol{\eta}_t, \quad (11)$$

$$\mathbf{c}_{t+1} = \gamma(a_t) \mathbf{c}_t + \mathbf{u}_t(\phi_t, \eta_{\text{reg}}) + \boldsymbol{\zeta}_t, \quad (12)$$

where Δ_s is the archetype specific degradation drift (Appendix C.1.2), $R(a_t) \in \{0, 1\}^{K \times K}$ is the diagonal restoration mask of action a_t applied to the components it targets with per component magnitude ρ_{mech} , $\gamma(a_t) \in [0, 1]$ is the fouling decay factor of the action (equal to 1 for non cleaning actions and to a small value for cleans), $\mathbf{u}_t$ is the region and phase dependent fouling source, and $\boldsymbol{\eta}_t, \boldsymbol{\zeta}_t$ are zero mean Gaussian noise terms that collect into the single noise symbol ξ_t of equation 9.

Phantom observation map (TurboSens2 only). Sensors do not observe ω_t directly. In TurboSens2, the observation map g first collapses the two wear channels through a phantom state

$$\tilde{\mathbf{s}}_t = \text{clip}(\mathbf{s}_t + M \max(\mathbf{0}, -\mathbf{s}_t) + C \mathbf{c}_t, s_{\min}, s_{\max}), \quad (13)$$

and then evaluates the OpenDeckSMR thermodynamic simulator at the current flight phase:

$$\mathbf{o}_t = g(\omega_t) + \boldsymbol{\varepsilon}_t = g_{\text{ODS}}(\tilde{\mathbf{s}}_t, \phi_t) + \boldsymbol{\varepsilon}_t, \quad \mathbf{o}_t \in \mathbb{R}^{S \times C}, \quad (14)$$

with $M \in \mathbb{R}^{K \times K}$ a fixed sign reversal mask allowing specific failure modes to manifest as apparent improvement, $C \in \mathbb{R}^{K \times K_c}$ a fouling to wear coupling matrix, and g_{ODS} the thermodynamic observation map evaluated across C flight phase contexts simultaneously. In TurboSens1 there is no fouling channel and $\tilde{\mathbf{s}}_t = \mathbf{s}_t$, so the observation reduces to $\mathbf{o}_t = g_{\text{ODS}}(\mathbf{s}_t, \phi_t) + \boldsymbol{\varepsilon}_t$.

Action dependent observability. The equations above imply that the observability of $\mathbf{s}_t$ is itself a function of the action history. Immediately after a cleaning action, $\mathbf{c}_{t+1} \approx 0$ so $\tilde{\mathbf{s}}_{t+1} \approx \mathbf{s}_{t+1}$ and the sensor stream reveals the true mechanical state; between cleans, fouling rebuilds and $\tilde{\mathbf{s}}_t$ diverges from $\mathbf{s}_t$, obscuring it. A representation that regresses directly on the phantom is mis specified; a world model that conditions on the action history can in principle identify both channels by comparing observations before and after clean events.

B Full Notation

C Description of the Environments

In this section we cover in details the proposed datasets.

Symbol	Description
<i>POMDP latent state</i>	
$\omega_t \in \Omega$	Full POMDP latent state at time t
$\mathbf{s}_t \in \mathbb{R}^K$	Engine mechanical wear (probing target)
$s_t^k \in [-b_k, 0 + \epsilon]$	Wear of component k
b_k	Per component degradation bound
$\mathbf{c}_t \in \mathbb{R}_{\geq 0}^{K_c}$	Fouling channel (TurboSens2 only)
$\tilde{\mathbf{s}}_t$	Phantom state, $\tilde{\mathbf{s}}_t = g_\phi(\mathbf{s}_t, \mathbf{c}_t)$
w_t	Weather state (ambient ΔT , precipitation gate)
$\phi_t \in \mathcal{C}$	Flight phase context
η_{reg}	Episode constant region and archetype label
$e_t \in \{0, 1\}^{n_{\text{evt}}}$	Stochastic event flag vector
K, K_c	Mechanical and fouling dimensions ($K=10$)
S	Number of sensor types (7)
C	Number of flight phase contexts (4 / 12 / 16)
n_{ch}	Number of latent channels in ω_t
<i>Actions and dynamics</i>	
$a_t \in \mathcal{A}$	Maintenance action at time t
$\Lambda_k(a_t)$	Per channel multiplicative action factor
$\lambda_{k,j}(a_t)$	Independent modulation j on channel k , $\Lambda_k = \prod_j \lambda_{k,j}$
$R(a_t) \in \{0, 1\}^{K \times K}$	Diagonal restoration mask of action a_t
ρ_{mech}	Per component restoration magnitude
$\gamma(a_t) \in [0, 1]$	Fouling decay factor
κ	Restoration quality coefficient
ξ_t	Process noise collecting ($\eta_t, \zeta_t, \dots$)
T	Episode length (variable)
<i>Observations and probing</i>	
$\mathbf{o}_t \in \mathbb{R}^{S \times C}$	Sensor observation tensor at time t
g, g_{ODS}	Observation map; OpenDeckSMR thermodynamic map
$M \in \mathbb{R}^{K \times K}$	Sign reversal mask in the phantom state
$C \in \mathbb{R}^{K \times K_c}$	Fouling to wear coupling matrix
ϵ_t	Observation noise
$\mathbf{z}_t \in \mathbb{R}^d$	Encoder embedding, $\text{Enc}_\theta(\mathbf{o}_{\leq t}, a_{\leq t})$
$\mathbf{z}_t^{\text{agg}} \in \mathbb{R}^d$	Temporally aggregated embedding
$\hat{\mathbf{s}}_{t+\tau}$	Probe prediction at horizon τ
h_ψ	Probe head, $\hat{\mathbf{s}}_{t+\tau} = h_\psi(\mathbf{z}_t)$
d	Embedding dimension
w	Encoder temporal window size
H	Predictor history (context) length
seq_len	Context length of the downstream target probe
λ	SIGReg regularisation weight

Table 5: Full notation used throughout the paper. The phantom state $\tilde{\mathbf{s}}_t$ is defined by equation 13.

480 C.1 The Scenario 1 Turbofan Degradation Dataset

481 We introduce a synthetic turbofan engine degradation dataset designed to benchmark world-model
482 and representation-learning methods under conditions of partial observability, heterogeneous failure
483 modes, and realistic maintenance interventions. Unlike existing turbofan benchmarks such as
484 CMAPSS [16], which expose only a single degradation trajectory with scalar health indicators,
485 Scenario 1 provides a *full ground-truth hidden state* alongside structured sensor observations, enabling
486 the direct evaluation of latent-state recovery rather than remaining-useful-life regression alone.

487 The released TurboSens1 dataset comprises 400 episodes of 10,000 timesteps each, totalling 4×10^6
488 timesteps; the generator’s maximum episode length cap is reached by all released episodes. Each
489 timestep represents one flight cycle. At every step the simulator records a ten dimensional hidden
490 degradation state $\mathbf{s}_t \in \mathbb{R}^{10}$, a sensor observation tensor $\mathbf{o}_t \in \mathbb{R}^{7 \times 12}$, a discrete maintenance action
491 $a_t \in \{0, \dots, 5\}$, a stochastic event mask, and weather covariates. The hidden state is available at
492 training time as a supervised signal and at evaluation time for probing learned representations; it is
493 never used as an input to the agent.

494 C.1.1 State and Observation Spaces

495 **Hidden health state.** The state $\mathbf{s}_t = (s_t^1, \dots, s_t^{10})^\top$ encodes the degradation of five turbofan
496 sub-systems, each represented by an efficiency deviation and a corrected flow deviation:

$$s_t^k \in [-b_k, 0], \quad k = 1, \dots, 10, \quad (15)$$

497 so that $s_t^k = 0$ denotes a pristine component and $s_t^k = -b_k$ corresponds to the OpenDeckSMR
498 airworthiness bound for that dimension, where b_k is component-specific. The five sub-systems are:
499 Booster (indices 1, 2), Fan (3, 4), High-Pressure Compressor (HPC; 5, 6), High-Pressure Turbine
500 (HPT; 7, 8), and Low-Pressure Turbine (LPT; 9, 10).

501 **Initial state and manufacturing variability.** Each episode initialises the health state as $\mathbf{s}_0 \sim$
502 $\mathcal{N}(\mathbf{0}, \sigma_0^2 \mathbf{I})$, truncated to $[-b_k/4, 0]$ per component to avoid initialising engines already near their
503 airworthiness limits. This reflects the empirical observation that turbofan engines exhibit measurable
504 component-level discrepancies in efficiency and flow capacity directly out of the factory, before any
505 degradation has accumulated. The dispersion σ_0 is calibrated so that the initial spread accounts for
506 roughly 5–10% of the per-component degradation budget, consistent with manufacturer acceptance
507 test data.

508 **Sensor observations.** At each timestep the physics simulator is queried at twelve fixed flight
509 contexts (see Section C.1.5), yielding an observation matrix $\mathbf{o}_t \in \mathbb{R}^{7 \times 12}$. The seven sensor channels
510 are listed in Table 6. The multi-context structure is deliberate: different flight conditions excite
511 different parts of the thermodynamic cycle, so the twelve slices of $\mathbf{o}_t$ together carry more information
512 about $\mathbf{s}_t$ than any single context in isolation. A world model that learns to aggregate evidence across
513 contexts is therefore rewarded.

Table 6: Sensor channels in $\mathbf{o}_t \in \mathbb{R}^{7 \times 12}$.

Index	Name	Description
0	HPC_Tout	HPC outlet total temperature
1	HP_Nmech	HP shaft rotational speed
2	HPC_Tin	HPC inlet total temperature
3	LPT_Tin	LPT inlet total temperature
4	Fuel_flow	Fuel mass flow rate
5	HPC_Pout_st	HPC outlet static pressure
6	LP_Nmech	LP shaft rotational speed

514 C.1.2 Degradation Dynamics

515 **Burn-in and steady drift.** Component degradation follows a two-regime model. In the early life of
516 each component, a rapid non-linear wear phase (burn-in) is superimposed on a persistent linear drift.
517 The incremental update at each step is

$$\Delta s_t^k = -\beta_k \exp\left(-\frac{t_k}{\tau_k}\right) - \alpha_k^{(m)} + \varepsilon_t, \quad (16)$$

518 where t_k is the time since the last repair of component k , $\beta_k > 0$ is the burn-in amplitude, τ_k is the
519 burn-in time constant sampled from $\mathcal{N}(500, 80^2)$ steps, $\alpha_k^{(m)}$ is the drift rate under speed regime
520 $m \in \{\text{slow, normal, fast}\}$, and $\varepsilon_t \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is per-step Gaussian noise. Integrating (16) from a
521 pristine component gives

$$\Delta s^k(T) = \beta_k \tau_k \left(1 - e^{-T/\tau_k}\right) + \alpha_k T, \quad (17)$$

522 which is calibrated so that a typical engine consumes roughly 70 to 80 percent of its degradation
523 budget over a 10,000-step episode. Speed regimes are resampled independently per component
524 every 500 steps according to per-component Categorical priors, introducing temporally correlated but
525 heterogeneous degradation rates across the state vector.

Degradation archetypes. To model fleet heterogeneity, each episode is assigned one of four failure-mode archetypes at generation time (Table 7). An archetype modulates α_k and β_k via per-component multipliers, concentrating wear in the sub-system that characterises the failure mode. Archetypes A, B and C may have a delayed onset: with probability 0.30 the failure mode is active from the first step; otherwise, onset is drawn uniformly from the interval [5%, 35%] of the episode length, before which the engine degrades as a D-balanced engine.

Table 7: Degradation archetypes and their dominant sub-systems.

ID	Name	Global scale	Dominant degradation
A	Compressor-driven	$1.0\times$	HPC and Booster (α up to $2.8\times$ base); models fouling and erosion in the hot section
B	Fan/Booster-driven	$1.0\times$	Fan and Booster (α up to $2.5\times$); models leading-edge erosion
C	Turbine-driven	$1.0\times$	HPT and LPT (α up to $2.8\times$); models thermal fatigue
D	Balanced	$0.65\times$	All components degrade at 65% of base rate; long-lived engine

The training split uses prior probabilities $\mathbb{P}(A) = 0.35$, $\mathbb{P}(B) = \mathbb{P}(C) = 0.25$, $\mathbb{P}(D) = 0.15$, reflecting the empirical distribution of failure modes in commercial turbofan fleets. An episode terminates early if any component reaches 97% of its degradation bound, modelling an airworthiness limit. In the generated dataset this end-of-life criterion is triggered in 91.7% of A-type episodes, 60.2% of C-type, 38.4% of B-type, and only 0.7% of D-type episodes, confirming that D engines typically exhaust the maximum episode length rather than failing catastrophically.

C.1.3 Maintenance Policy and Actions

Six discrete maintenance actions are available (Table 8). The expert policy is an oracle with imperfect execution, designed to produce action sequences that are strongly correlated with the hidden health state while still leaving room for improvement by a learned agent.

Life-Limiting-Part (LLP) ceilings. Each of the ten health-state components is associated with a fleet-wide, regulation-level cycle limit L_k that is identical across archetypes, modelling international airworthiness requirements. A per-component counter c_k is incremented each step and reset to zero when the component is replaced. Whenever $c_k = L_k$ for any component, a maintenance visit is forced before the next step, irrespective of the current health state. LLP ceilings therefore act as a hard upper bound on the inter-maintenance interval and are state-independent by design.

Health-correlated proactive scheduling. On top of the LLP ceiling, the policy estimates for each component the predicted time until either the LLP cycle limit or an internal health threshold (a fraction of the degradation bound, kept conservative so that airworthiness margins are never approached) is reached, based on the current instantaneous degradation rate. The trigger is then modulated by the operational calendar: commercial operators try to minimise scheduled visits during peak traffic windows so that engines are not removed from service while the airline most needs them, and prefer to bring forward a borderline visit into the surrounding shoulder season when it is operationally safe to do so. Concretely, a flight that lands inside a high-demand window is more likely to defer a borderline-but-safe visit and a flight that lands outside it is more likely to bring one forward, conditional on the engine condition permitting it; the airworthiness margin remains intact in either case. Because the predicted time depends on the degradation rate, which is itself governed by archetype, speed regime and recent event history, the timing of health-triggered visits is correlated with the latent state, with the calendar acting as a soft secondary modulator.

Min-built opportunistic bundling. Whenever a maintenance visit is triggered for any reason (LLP, health, or deferred repair-driven, see below), the policy scans all remaining components and applies min-built logic: if component k is predicted to need replacement within a min-built window $W_{mb} \in [2000, 3000]$ steps under its current degradation trajectory, it is serviced during the same visit. Because an engine shop visit has a large fixed downtime cost in practice (on the order of months), bundling is strongly preferred over revisiting. If the bundle spans two of the three engine modules,

the recorded action is promoted to `full_overhaul` rather than leaving one module idle during the downtime.

Imperfect restoration. The restoration applied at a maintenance visit is imperfect even for LLP-mandated replacements. A newly installed component is mounted inside an ageing engine environment whose bearings, seals, casings and neighbouring modules continue to wear, so no action restores the engine to a pristine overall state. Concretely, an LLP replacement restores the targeted component by a fraction drawn from a tight distribution centred around 0.80 to 0.90 of its degradation budget; a repair action restores by a left-skewed fraction in the ranges given in Table 8, reflecting the variability of human intervention quality. A manufacturing-quality coefficient $\kappa \sim \mathcal{U}(0.70, 1.00)$ multiplies the restoration uniformly across all visits of an episode, capturing engine-to-engine variability in shop quality.

Deferred repairs. During each maintenance visit, every component that was inspected but not serviced has a small independent probability (3 to 7%) of being a *deferred repair*: an inspection miss in which the component was mis-assessed and is actually degraded beyond the threshold for replacement. A deferred repair is not detected immediately. After a stochastic delay of $[200, 800]$ steps, the engine is recalled for an unplanned emergency visit that services the missed component. The total number of deferred repairs accumulated across an episode is stored in the metadata. Deferred repairs are a deliberate source of sub-optimality in the expert policy, giving downstream learning methods a concrete target for improvement.

Interval asymmetry. The combination of LLP ceilings, health-correlated scheduling, min-built bundling and deferred repairs produces an inter-maintenance interval distribution that is strongly asymmetric in the dataset. The LLP ceiling imposes a sharp upper cutoff, the health trigger concentrates most visits in a band that depends on the archetype-specific degradation rate, and deferred repairs populate a short-interval tail via forced emergency visits. This is intentionally different from the earlier symmetric uniform scheduling used in our prior datasets, in which action timing was statistically independent of the hidden state and therefore provided no signal for horizon prediction.

Table 8: Maintenance actions and the components each one services.

ID	Name	Components serviced
0	<code>do_nothing</code>	None
1	<code>fan_overhaul</code>	Fan Eff, Fan Wc
2	<code>hpc_overhaul</code>	Booster Eff/Wc, HPC Eff/Wc
3	<code>turbine_overhaul</code>	HPT Eff/Wc, LPT Eff/Wc
4	<code>full_overhaul</code>	All ten components
5	<code>targeted_patch</code>	Most-degraded component (oracle)

Extended action metadata. To carry the richer information produced by the LLP and min-built logic, each recorded action is accompanied by a metadata vector containing: a ten-dimensional boolean mask of components actually serviced during the visit (LLP and min-built inclusions expand this mask beyond the nominal action target); a categorical flag distinguishing LLP-forced, health-triggered, and deferred repair-forced visits; and the per-visit deferred repair count. This preserves the discrete six-action label used by the RL environment while exposing the full per-component servicing information needed by diagnostics and representation-learning probes.

C.1.4 Stochastic Event Library

The probabilities and magnitudes of these events are not calibrated to real-world aircraft failure rates and do not represent specific failure modes observed in operational fleets. Although the events draw inspiration from real engine phenomena, their probabilities and magnitudes are mathematical abstractions and in no case reflect what is observed in real life flight settings. They are included to make the latent state recovery problem harder and the encoder representations more challenging to learn, by injecting sparse, structured discontinuities into the trajectories of $\mathbf{s}_t$, $\mathbf{c}_t$ or $\mathbf{o}_t$. Calibrating the events to operational data, or implementing physically grounded event models more broadly, is left for future work. We accordingly use abstract effect typed labels in the paper

(perm_step_*, trans_anomaly_*, etc.); the released simulator currently uses different (more suggestive) code labels which will be aligned with the abstract scheme in a subsequent release.

Ten categories of stochastic events enrich the dataset (Table 9). Events are drawn from independent Poisson processes. Permanent events (perm_step_1, perm_step_2, perm_drift_1, perm_drift_2) apply an immediate step change or sustained drift increment to one or more components of s_t . Transient events (trans_anomaly_1, trans_anomaly_2, trans_drift_1, trans_drift_2, trans_fouling_1) inject a short-lived perturbation that decays back toward baseline. Sensor-level events (sensor_pulse_1) perturb the observation tensor $\mathbf{o}_t$ without modifying the latent state, deliberately introducing observation noise that is not explained by (s_t, c_t) and thus testing a world model’s ability to distinguish structural degradation from transient sensor artefacts. The overall stochastic event rate across the training set is 0.77% of timesteps. Some events are phase-gated to low-altitude contexts and others additionally require a cold ambient temperature below a threshold derived from the ISA standard lapse rate.

Table 9: Stochastic event library: abstract label, type, and mean Poisson rate per 10,000 steps. Rates and effect magnitudes are not calibrated to any real-world failure statistics.

Event	Type	Rate / 10k	Effect
perm_step_1	Permanent	0.1	Single-component step decrement on s
perm_step_2	Permanent	0.3	Step decrement on a different component group
perm_drift_1	Permanent	0.1	Slow multi-component drift increment
perm_drift_2	Permanent	0.3	Single-channel sustained drift
trans_anomaly_1	Transient	1.0	Short pulse on shaft-speed channels
trans_anomaly_2	Transient	0.5	Short pulse with elevated drift
trans_drift_1	Transient	1.5	Multi-step drift-rate boost
trans_drift_2	Transient	0.3	Phase-gated drift modulator (cold contexts)
trans_fouling_1	Transient	0.2	Multi-step bias on fouling channel
sensor_pulse_1	Sensor	1.0	Instantaneous outlier on one sensor channel

C.1.5 Flight Contexts and Weather Process

Each step is observed under $C = 12$ fixed flight contexts spanning three operating phases: six cruise conditions, three climb conditions, and three takeoff conditions. The contexts vary along four axes: altitude (0 to 37,000 ft), Mach number (0 to 0.82), ambient temperature deviation ΔT_{amb} (from -5 K to $+20$ K above ISA), and thrust command. This design ensures that sensors associated with the compressor path (HPC_Tout, HP_Nmech) are strongly excited during takeoff, while cruise contexts provide the most sensitive signal for turbine-path sensors, making context selection a non-trivial component of inference.

The weather process models temporal correlations in ΔT_{amb} via an Ornstein-Uhlenbeck (OU) process,

$$d(\Delta T) = \theta(\mu - \Delta T) dt + \sigma dW_t, \quad (18)$$

generating slowly varying ambient conditions with a characteristic mean-reversion timescale of roughly 25 steps (confirmed by the empirical autocorrelation function). Rain is modelled as a Bernoulli gate that opens only when ΔT_{amb} is below a threshold, introducing correlation between precipitation and cold-ambient contexts. The episode-level rain rate averages approximately 27%.

C.1.6 Dataset Statistics and Health Assessment

Table 10 summarises the key quantitative properties of the training split.

All released episodes run to the 10,000 step cap; archetype D (balanced) under represents the failure modes that drive earlier termination, but with the maintenance policy in place, even the heavy archetypes A–C are kept within airworthiness limits and reach the maximum length. Users who want longer trajectories for forecasting can produce them with the same configuration.

To characterise the separation between health regimes in the hidden state space, we assign a binary health label to each timestep: a timestep is *healthy* if it falls within the first 300 steps of an episode

Table 10: Scenario 1 released training split: summary statistics.

Property	Value
Episodes	400
Total timesteps	4×10^6
Episode length	10,000 steps (all episodes at the cap)
Archetype counts (A / B / C / D)	121 / 104 / 102 / 73
NaN or Inf values in states	0
NaN or Inf values in sensors	0
Non zero maintenance rate	0.04% of timesteps
Stochastic event rate	<1% of timesteps
Sensor observation shape	$(N, 7, 12)$

or within 300 steps of a non-zero maintenance action, and *degraded* if it lies beyond 55% of the episode lifetime and is not covered by a healthy window. Figure ?? shows the resulting overlapping marginal distributions for all ten state dimensions, with Cohen’s d reported per dimension. Separation is pronounced for HPC and turbine efficiency and flow dimensions ($d \approx 1.5$ to 2.0), confirming that the hidden state carries a strong and consistent signal distinguishing healthy from worn engines. The bleed_error and nac_drag dimensions exhibit moderate overlap, reflecting their weaker coupling to the dominant failure modes in archetypes A through C.

C.2 TurboSens2: Regional Operating Environments and Fouling Dynamics

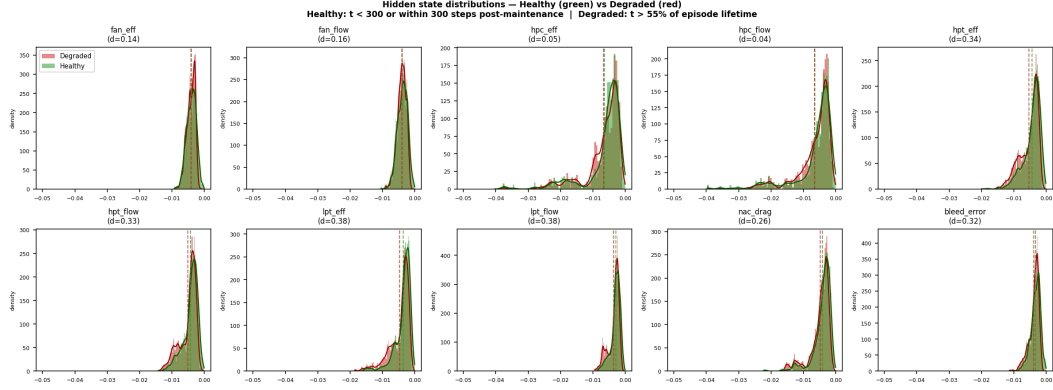
TurboSens2 extends the Scenario 1 dataset along three physical axes that are absent from existing turbofan benchmarks: the operating environment in which an engine spends its service life, the separation between true mechanical wear and reversible fouling of the gas path, and the cross-module aerodynamic coupling that causes a locally damaged component to make its neighbours appear healthier than they are. The scenario retains the core ingredients of Scenario 1 (heterogeneous degradation archetypes, life-limiting-part regulatory constraints, an imperfect expert maintenance policy, a library of stochastic events, multi-context sensor observations and a fully exposed hidden state) and layers on top of them a regional operating model, a two-channel hidden state, a seasonal weather process with realistic shop-visit downtimes, and an additional maintenance action capturing periodic engine water washing.

The dataset is organised around six region archetypes, each representing a distinct operating environment characterised by its dominant atmospheric stressor (sand, salt, humidity, pollution, thermal swing). A region archetype simultaneously biases the prior over degradation archetypes, scales the mechanical drift rates of the most exposed sub-systems, governs the fouling accumulation rate, restricts the library of admissible stochastic events, and modulates the seasonal weather process and the flight cadence. Between 80 and 400 episodes are generated per region archetype, for a target total of approximately 1,500 episodes with a mean length of 14,000 timesteps and a hard ceiling of 20,000 timesteps.

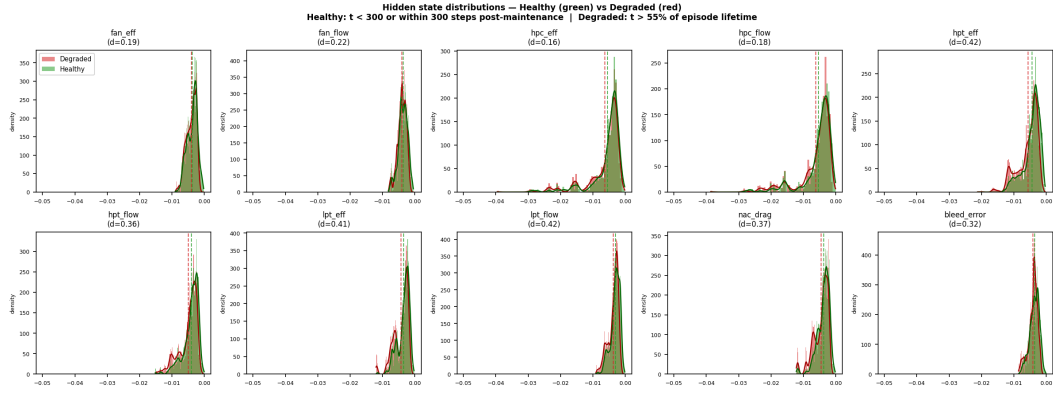
C.2.1 Two-Channel Hidden State

Mechanical and fouling channels. The Scenario 1 hidden health state $\mathbf{s}_t \in \mathbb{R}^{10}$ is preserved and interpreted strictly as the *mechanical* deviation of efficiency and corrected flow from nominal for the five sub-systems (Booster, Fan, HPC, HPT, LPT). Its evolution is driven solely by slow stochastic wear (archetype-specific drift, burn-in transient, and speed-regime noise); it is irreversible except through partial restoration at shop visits. To capture the fast, reversible fouling of the gas path we introduce a second hidden channel $\mathbf{c}_t \in [0, 1]^5$ encoding a normalised fouling level per sub-system. Fouling accumulates each flight at a rate conditioned on region, flight phase and active stochastic event set, and resets to zero after any engine-wash action. Crucially, $\mathbf{c}_t$ does *not* enter the mechanical state update: $\mathbf{s}_t$ and $\mathbf{c}_t$ evolve on independent dynamics, representing two physically distinct phenomena (material wear vs. surface deposition).

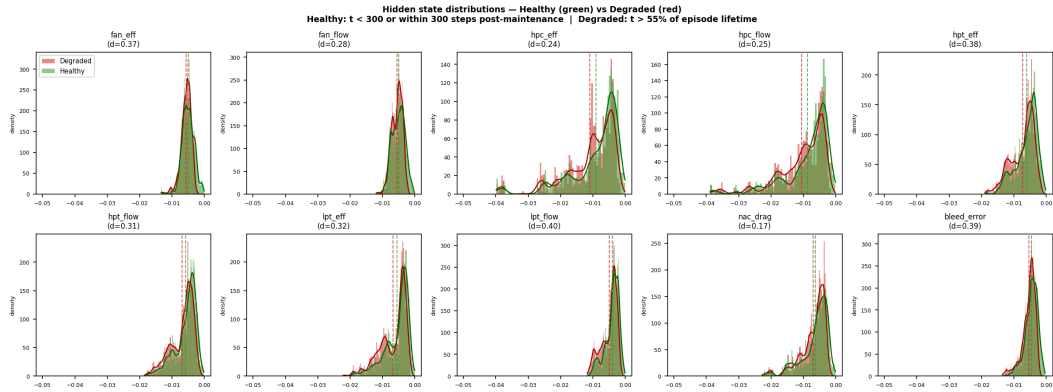
Observation model: the phantom state. The thermodynamic simulator produces sensor readings from an engine state vector, so every hidden effect that modifies the gas-path flow must enter the



(a) Train set underlying state distribution between healthy and degraded.



(b) Test set in-distribution with train, between healthy and degraded.



(c) Test-hard set out-of-distribution

Figure 3: Distribution for Scenario 1 of the ten health-state components under healthy (blue) and degraded (orange) regimes. Train and test share the same distribution; test-hard introduces stronger degradation, varied drift rates, and modified maintenance to assess generalization beyond the training manifold.

simulator through that vector. We therefore define a *phantom state* $\tilde{s}_t \in \mathbb{R}^{10}$ that aggregates, on top of s_t , the two observable-but-hidden effects that the simulator cannot model intrinsically:

1. *Phantom flow coupling*: a breach or severe local wear redirects airflow through neighbouring modules, producing corrected-flow readings that are anomalously high for the healthy-looking neighbour while the true cause lies elsewhere.
2. *Fouling coupling*: surface fouling perturbs module efficiency and apparent flow even when the underlying mechanical state is pristine.

Both effects are implemented as additive, non-negative couplings clipped to the validated thermodynamic operating range:

$$\tilde{s}_t^k = \text{clip}\left(s_t^k + \sum_{j=1}^{10} M_{kj} \max(0, -s_t^j) + \sum_{m=1}^5 C_{km} c_t^m, \underline{b}_k, \bar{b}_k\right), \quad k = 1, \dots, 10, \quad (19)$$

where $M \in \mathbb{R}^{10 \times 10}$ is the sparse phantom flow-coupling matrix of Scenario 1 (strictly positive entries only on flow dimensions immediately downstream of each degrading efficiency component), and $C \in \mathbb{R}^{10 \times 5}$ is a new sparse, non-negative fouling-to-phantom matrix whose entry C_{km} is non-zero only when fouling of module m physically perturbs state dimension k (for example, HPC fouling degrades HPC efficiency and biases HPC flow upward). The mechanical-state bounds $[\underline{b}_k, \bar{b}_k]$ are those used by the underlying simulator: strictly non-positive on efficiency channels, and admitting positive flow deviations up to +0.03 for Fan, Booster and HPC flow and +0.05 for HPT and LPT flow. Coupling weights in M and C are calibrated empirically to keep $\tilde{s}_t$ within validated thermodynamic ranges under all archetype and region combinations and are exposed as named constants to support ablation studies; the matrices are fixed across the dataset while s_t and c_t vary per episode and timestep.

Final observation map (TurboSens2). This is the observation map for TurboSens2 only; in TurboSens1 the fouling channel is absent, $\tilde{s}_t$ collapses to s_t , and the simulator is queried directly on the mechanical state. In TurboSens2, the simulator is queried on $\tilde{s}_t$ rather than s_t , so fouling and phantom coupling both reach the sensors through the same physically grounded pathway, the state vector that the thermodynamic model actually consumes:

$$\mathbf{o}_t = g(\tilde{s}_t, \omega_t) + \varepsilon_t, \quad (20)$$

where g is the OpenDeckSMR thermodynamic model, ω_t collects the ambient and command conditions of the flight context, and ε_t is residual sensor noise. Because c_t enters the observation exclusively through $\tilde{s}_t$ and is reset by the engine-wash action while s_t is not, a wash produces the characteristic sawtooth pattern in the sensor stream while the underlying mechanical trajectory s_t is unaffected. Both c_t and the phantom contribution $\tilde{s}_t - s_t$ are exposed in the dataset metadata as per-step ground-truth labels, enabling supervised evaluation of disentangling methods.

C.2.2 Region Archetypes

Each episode is assigned one of six region archetypes at generation time (Table 11). The names are deliberately environmental rather than geographic. A region archetype determines four generation-time parameters: (i) a prior over the four Scenario 1 degradation archetypes, (ii) a per-subsystem mechanical drift scaling factor, (iii) a baseline fouling accumulation rate, and (iv) a mean flight cadence controlling how many simulated flights map to one calendar day.

Region-conditional archetype prior. The four Scenario 1 degradation archetypes (A compressor-driven, B fan/booster-driven, C turbine-driven, D balanced) are retained unchanged. Their prior probability is now conditioned on the region (Table 12), so that for instance Arid over-represents A (sand erosion of the HPC) and Littoral over-represents B (salt corrosion of the fan and booster).

Per-subsystem mechanical scaling. Independently of the archetype, the region applies a multiplicative scaling $\rho_k^{(r)} > 0$ to the drift rate α_k of equation (16), modelling the physical acceleration of wear by the regional stressor (Table 13). Littoral scales fan and booster drift, Arid scales HPC drift, Highland shifts stress toward the thermal section.

Table 11: Region archetypes in TurboSens2. Severity is a qualitative summary of combined mechanical and fouling load.

ID	Name	Dominant stressor	Severity
R1	Continental	None (baseline)	Benign
R2	Littoral	Salt spray, humidity	Moderate
R3	Arid	Sand, dust, thermal swing	Harsh
R4	Equatorial	Heat, humidity, bio-fouling	Moderate
R5	Highland	Cold ambient, thin air	Benign-Moderate
R6	Industrial	Chemical pollution, soot	Moderate-Harsh

Table 12: Conditional prior $\mathbb{P}(\text{archetype} \mid \text{region})$ used at episode generation.

Region	A	B	C	D
Continental	0.30	0.20	0.25	0.25
Littoral	0.20	0.40	0.20	0.20
Arid	0.45	0.30	0.15	0.10
Equatorial	0.30	0.30	0.20	0.20
Highland	0.20	0.15	0.35	0.30
Industrial	0.35	0.25	0.20	0.20

728 C.2.3 Degradation and Fouling Dynamics

729 **Extended mechanical increment.** The Scenario 1 degradation increment (16) is extended with two
730 region-conditional terms: the region scaling $\rho_k^{(r)}$ and a slow corrosion term driven by the accumulated
731 fouling,

$$\Delta s_t^k = -\beta_k e^{-t_k/\tau_k} - \rho_k^{(r)} \alpha_k^{(m)} - \lambda_k c_t^{\pi(k)} + \varepsilon_t, \quad (21)$$

732 where $\pi(k) \in \{1, \dots, 5\}$ maps a state component to its parent sub-system and λ_k is a small corrosion
733 coefficient set so that a sub-system held at $c = 1$ for an entire episode would contribute an additional
734 drift of the same order as 15 to 20 percent of the base archetype drift. Salt and sand therefore
735 accelerate mechanical wear only to the extent that cleaning is deferred.

736 **Fouling increment.** Fouling accumulates between maintenance visits at a rate modulated by the
737 region archetype and the flight phase of each of the sixteen snapshot contexts seen within a step:

$$\Delta c_t^\ell = \eta_\ell^{(r)} \sum_{i=1}^{16} w_{\phi_i} - \delta c_t^\ell, \quad (22)$$

738 where $\eta_\ell^{(r)}$ is the region-conditional deposition rate (listed last column of Table 13 after normalisation
739 by sub-system exposure), w_{ϕ_i} weights the contribution of each snapshot context by its flight phase
740 (takeoff and climb deposit more than cruise), and δ is a small self-cleaning term accounting for
741 aerodynamic blow-off. Fouling is clipped to $[0, 1]$ per sub-system.

742 Events (Section C.2.5) can additionally apply a large step increment to c_t (a `trans_drift_1` or
743 `regional_perm_1` event deposits a bulk increment on c_t in a single step) or, for events such as
744 `perm_step_1`, a joint step change to both s_t and c_t that is visible in the snapshot at the event step.
745 Abstract labels are used everywhere in the paper; the released simulator currently exposes legacy
746 code labels that will be aligned with the abstract scheme in a subsequent release.

747 **Phantom-coupling interpretation.** Because the phantom map (19) depends on the true state
748 s_t , an episode in which a single efficiency dimension degrades aggressively will exhibit a non-
749 physical improvement in the downstream flow dimension of a neighbouring module, followed by a
750 discontinuous return to nominal after maintenance restores the efficiency. This provides a genuine
751 counter-example to monotonic degradation heuristics and serves as a latent-structure probe for
752 representation learning: a world model that has learned the true mechanical state should detect the

Table 13: Region-induced mechanical drift multipliers on α_k and baseline fouling accumulation multiplier.

Region	HPC	Fan / Booster	HPT / LPT	Fouling rate
Continental	1.0	1.0	1.0	1.0×
Littoral	1.0	1.5	1.1	1.4×
Arid	1.6	1.3	1.0	1.8×
Equatorial	1.1	1.2	1.0	1.5×
Highland	0.9	0.9	1.2	0.7×
Industrial	1.2	1.2	1.1	1.6×

upstream efficiency loss as the cause of the downstream flow excursion, whereas a purely local model will mis-attribute the event.

C.2.4 Maintenance Policy and Extended Action Set

Engine water wash. A seventh maintenance action, `engine_wash`, is added to the original six Scenario 1 actions (Table 14). A wash resets the fouling vector $\mathbf{c}_t$ to zero (up to a small residual noise), does not restore mechanical state, and incurs only a short downtime (no multi-month calendar jump). Wash timings are not triggered by health; they follow a semi-periodic schedule per episode, with the mean wash interval drawn from $[50, 150]$ steps for benign regions and $[30, 90]$ steps for harsh regions, jittered by a per-wash noise term. This models the scheduled maintenance practice of most commercial operators and produces the sawtooth pattern in the observed sensor stream that our dataset is designed to expose.

Table 14: TurboSens2 maintenance actions. Actions 0 through 5 are inherited from Scenario 1; action 6 is the new engine wash.

ID	Name	Target
0	<code>do_nothing</code>	None
1	<code>fan_overhaul</code>	Fan Eff, Fan Wc
2	<code>hpc_overhaul</code>	Booster Eff/Wc, HPC Eff/Wc
3	<code>turbine_overhaul</code>	HPT Eff/Wc, LPT Eff/Wc
4	<code>full_overhaul</code>	All ten mechanical components
5	<code>targeted_patch</code>	Most-degraded component (oracle)
6	<code>engine_wash</code>	Reset $\mathbf{c}_t$ (fouling only)

Shop-visit downtime and calendar jumps. Unlike Scenario 1, TurboSens2 represents the wall-clock cost of a shop visit in the weather process rather than abstracting it away. Each non-wash maintenance visit advances the episode calendar clock by a duration drawn from a severity-dependent range: light targeted patches add 0.5 to 1 month, mid-weight module overhauls add 2 to 3 months, and heavy or extreme-degradation overhauls add 5 months. The seasonal weather state (Section C.2.7) jumps forward accordingly at the next step, producing abrupt ambient conditions transitions at the return from a shop visit that are a distinctive feature of fleet operations. Fouling is reset to zero at every maintenance visit, whether or not a wash action is formally issued.

Seasonal modulation of proactive scheduling. Scenario 2 makes the calendar trigger from Scenario 1 explicit through the seasonal weather process. The proactive scheduler now scales its borderline window by a seasonal factor $\kappa_{\text{season}}(t) \in [0.6, 1.3]$ that peaks in local summer (stronger deferral pressure) and is lowest in local winter (stronger anticipation pressure). Life-limiting-part ceilings and emergency thresholds are not subject to this seasonal modulation: airworthiness-mandated replacement remains inflexible whatever the season.

LLP and base policy structure. Life-limiting-part cycle limits are identical across all regions and archetypes, reflecting their regulatory origin rather than any physical susceptibility. Health-correlated scheduling, min-built bundling, deferred repairs and imperfect restoration behave exactly as in Scenario 1, operating on the mechanical state $\mathbf{s}_t$ and ignoring the fouling channel $\mathbf{c}_t$. The wash action

782 is handled as a separate track that never interacts with the LLP counter and never causes min-built
783 bundling.

784 C.2.5 Stochastic Event Library

785 The Scenario 1 event library is retained and extended with region based gates on admissible events.
786 `trans_drift_2` is excluded in Arid and Equatorial regions regardless of instantaneous weather, since
787 the ambient temperature gate that conditions it is never satisfied in those regions. `regional_perm_1`
788 is strongly over represented in Arid and to a lesser extent Industrial regions. `regional_drift_1`,
789 introduced in Scenario 2 as a new event type, is gated to Littoral and occasionally Equatorial. Events
790 that jointly affect s_t and c_t (`perm_step_1` and `regional_perm_1`) apply their joint step change at
791 the event timestep, so that the snapshot observation at that step already reflects the perturbed engine.

792 The event rate per region is calibrated to keep the global stochastic event rate below one percent
793 of timesteps, preserving the sparsity property that makes events challenging to detect. All event
794 metadata (type, timestep, affected components, magnitude) is exposed as ground-truth labels for
795 anomaly-detection and representation-probing tasks.

796 C.2.6 Flight Contexts

797 Each timestep is observed under $C = 16$ fixed flight contexts spanning three physically supported
798 phase types in OpenDeckSMR: cruise (CR), max-climb (MCL) and take-off (MTO). The breakdown
799 is eight nominal cruise contexts spanning altitudes from 33,000 to 39,000 ft and Mach numbers
800 from 0.76 to 0.84, one low-power cruise context at 15,000 ft approximating an idle approach
801 regime, four max-climb contexts covering nominal, stepped, hot and cold variants, and three take-off
802 contexts covering nominal, hot-day and heavy-weight variants. The low-power context is labelled
803 as a low-thrust proxy rather than a true descent phase because OpenDeckSMR does not validate
804 thermodynamic behaviour in the windmilling descent regime; its command value is reduced to the
805 lower bound of the CR envelope.

806 Each context remains internally consistent: take-off contexts combine sea-level altitude with high
807 ambient temperature and zero Mach, cruise contexts combine high altitude with low ambient temper-
808 ature and high Mach, climb contexts span intermediate values. Cross-contaminating context values
809 (for example a cruise DTAMB applied at take-off altitude) are explicitly forbidden by the sampler to
810 keep every snapshot inside the validated operating envelope of the physics simulator.

811 C.2.7 Seasonal Weather Process

812 TurboSens2 replaces the single-level Ornstein-Uhlenbeck weather process of Scenario 1 with a three-
813 level hierarchical generator that captures seasonality, flight-to-flight variability and within-snapshot
814 noise.

815 **Seasonal baseline.** A slow sinusoidal drift $\mu_{\text{season}}^{(r)}(d_t)$ indexed by the simulated calendar day
816 d_t models the region-specific annual cycle of ambient temperature. Its amplitude is largest for
817 Continental (approximately ± 25 K peak-to-peak around the regional mean), intermediate for Highland
818 and Industrial, and smallest for Equatorial (near-flat). The calendar day advances by $1/\nu^{(r)}$ per
819 timestep, where $\nu^{(r)}$ is the region-conditional mean flight cadence: Continental and Industrial at 4
820 to 5 flights per day, Littoral and Equatorial at 2 to 3, Arid and Highland at 1 to 2, with per-region
821 jitter. Maintenance downtimes advance the calendar by the severity dependent amount described in
822 Section C.2.4.

823 **Flight-level OU.** Around the seasonal baseline, a faster OU process governs the weather realised
824 for one flight,

$$d(\Delta T_{\text{flight}}) = \theta_f (\mu_{\text{season}}^{(r)}(d_t) - \Delta T_{\text{flight}}) dt + \sigma_f dW_t, \quad (23)$$

825 with θ_f chosen so the mean-reversion half-life covers several flights and σ_f scaled to produce a
826 realistic day-to-day ambient dispersion. A fresh realisation is drawn at the start of each simulated
827 flight so that consecutive snapshots within a flight share correlated weather, while consecutive flights
828 can differ substantially.

Within-snapshot jitter. A third small-amplitude Gaussian perturbation is finally added to each of the sixteen snapshot contexts within a step, in addition to the context-specific offsets documented in Section C.2.6. This captures turbulence and small-scale atmospheric variability that differentiates the snapshots even under an otherwise stable flight-level OU.

C.2.8 Extreme-Degradation Episodes

A small fraction of episodes (between 1 and 5 percent) are flagged as extreme-degradation episodes at generation time. In these episodes the proactive policy’s planning window and emergency threshold are temporarily relaxed, allowing one or more components to traverse a deep sag down to 92 to 95 percent of the degradation bound before a heavy overhaul is dispatched. The overhaul restores the affected components to between 88 and 95 percent of the pristine bound, resets fouling, and advances the calendar clock by five months. Extreme-degradation episodes are labelled and carry metadata describing the sag window, the components involved, the restoration quality and the post-visit calendar state. They are deliberately rare so that reliable engines remain the statistical norm, but are a vital stress test for prognostic methods that must gracefully handle the upper tail of the degradation distribution.

C.2.9 Ground-Truth Metadata

Every TurboSens2 episode carries a dense set of ground-truth labels exposed alongside the sensor stream, making the dataset usable for self-supervised pretraining, probing, anomaly detection and fine grained diagnostic studies:

- Region archetype label $r \in \{R1, \dots, R6\}$ and degradation archetype label $\in \{A,B,C,D\}$.
- Mechanical state trajectory $\mathbf{s}_t \in \mathbb{R}^{10}$ and fouling trajectory $\mathbf{c}_t \in \mathbb{R}^5$ at every step.
- Phantom-coupled state $\tilde{\mathbf{s}}_t$ and the coupling matrix M used to generate it.
- Action label $a_t \in \{0, \dots, 6\}$ including the new wash action, together with the Scenario 1 extended action metadata (component-service mask, visit-type flag, deferred repair count).
- Stochastic event mask and per-event parameters.
- Simulated calendar day d_t , season label, and flag indicating whether the step follows a shop-visit calendar jump.
- Flight cadence $\nu^{(r)}$ realised for the episode, wash-schedule timings, and an extreme-degradation boolean flag.

C.2.10 Dataset Statistics

Table 15 reports the observed statistics of the three released TurboSens2 splits. The train split contains 500 episodes, the two evaluation splits contain 60 episodes each (drawn with held-out seeds, identical generator), and all three splits use the same region and archetype priors.

Table 15: TurboSens2 dataset statistics, measured on the released HDF5 files.

Property	Train	Test	Test-hard
Episodes	500	60	60
Total flights	7,010,872	830,756	854,710
Mean episode length (steps)	14,022	13,846	14,245
Min / max episode length	8,000 / 20,000	8,000 / 20,000	8,000 / 20,000
Extreme-degradation episodes	2.8%	5.0%	6.7%
Engine-wash fraction	1.052%	1.044%	1.044%
Repair fraction (non-wash)	0.093%	0.094%	0.090%
of which proactive	0.085%	0.085%	0.083%
of which deferred repair-forced	0.003%	0.003%	0.003%
of which re-repair	0.004%	0.005%	0.004%

All three splits exhibit a healthy 100% valid-flight rate, reflecting the context-envelope hardening introduced in Scenario 2 (see §C.2.6); no OpenDeckSMR convergence failures occur in the released data. Repair actions are dominated by proactive visits, consistent with the health-correlated

865 maintenance policy: emergency visits are essentially absent because the maintenance scheduler
866 intervenes before components exceed the airworthiness threshold. Engine-wash events fire roughly
867 once per 100 flights, which amounts to about ten wash cycles per average-length episode and gives
868 the world model meaningful opportunities to observe the phantom-to-true transition.

869 **Region and archetype coverage.** Table 16 shows the joint region $\times$ archetype episode distribution
870 for the train split. The region prior is skewed toward Industrial (97) and Highland (91) and away
871 from Arid (70) and Littoral (71); the archetype prior is dominated by A_compressor (179) and
872 B_fan_booster (128). The region-conditional archetype prior described in §C.2.2 produces the
873 expected concentrations (Arid is the only region where A_compressor reaches 40 of 70, reflecting
874 HPC-fouling bias; Highland reaches 42 turbine episodes out of 91). Figure 4 visualises the full
875 heatmap for all three splits.

Table 16: Region and archetype marginal counts for the TurboSens2 train split.

Region	# Episodes	Archetype	# Episodes
Continental	84	A_compressor	179
Littoral	71	B_fan_booster	128
Arid	70	C_turbine	121
Equatorial	87	D_balanced	72
Highland	91		
Industrial	97		

876 **Event rates by region.** Table 17 reports the rising-edge event rates per ten thousand flights on the
877 train split. The two *regional* events (`regional_perm_1`, `regional_drift_1`) fire orders of magni-
878 tude more often than the global event library and carry strong regional gating: `regional_perm_1`
879 reaches 898 firings per ten thousand flights in Arid and 339 in Industrial while dropping to 31 in
880 Highland, and `regional_drift_1` is concentrated in Littoral, Equatorial and Industrial while being
881 fully gated off in Arid and Highland. The global events retain order-of-magnitude rates of 0.1 to 2.7
882 per ten thousand flights across all regions, matching the scenario 1 rate envelope. Figure 8 visualises
883 the corresponding per-flight activity matrix.

Table 17: Event rising-edge rate per 10^4 flights, TurboSens2 train split. Two rightmost columns show the two regional-only events; all other rows are the scenario 1 event library. Rates are sampling artefacts of the generator and not calibrated to real failure statistics.

Event	All	Cont.	Litt.	Arid	Equa.	High.	Ind.
<code>perm_step_1</code>	0.11	0.14	0.18	0.07	0.11	0.08	0.09
<code>trans_drift_1</code>	1.80	1.50	1.04	3.93	1.13	0.78	2.65
<code>trans_anomaly_1</code>	1.05	1.05	1.09	0.91	1.18	1.01	1.03
<code>perm_step_2</code>	0.33	0.31	0.23	0.37	0.40	0.37	0.29
<code>trans_fouling_1</code>	0.19	0.24	0.16	0.17	0.17	0.16	0.21
<code>trans_anomaly_2</code>	0.44	0.46	0.40	0.34	0.47	0.44	0.50
<code>perm_drift_1</code>	0.11	0.14	0.15	0.08	0.08	0.10	0.10
<code>perm_drift_2</code>	0.30	0.37	0.30	0.26	0.34	0.30	0.25
<code>sensor_pulse_1</code>	1.00	0.92	0.98	1.12	1.05	0.98	0.96
<code>trans_drift_2</code>	0.23	0.25	0.17	0.00	0.00	0.65	0.21
<code>regional_perm_1</code>	245.91	49.38	48.54	897.67	197.38	30.76	338.53
<code>regional_drift_1</code>	23.70	33.93	1.51	0.00	47.92	0.00	48.96

884 C.2.11 Sensor, Hidden-State and Fouling Marginals

885 The dataset ships with three empirical artefacts that together characterise the observation surface
886 of TurboSens2. Figure 5 overlays the marginal distribution of each of the seven DeckSMR sensors
887 across the sixteen flight-phase contexts, illustrating both the context separation in the observation
888 space and the inter-episode spread induced by the context-jitter mechanism described in §C.2.6.
889 Figure 6 contrasts the marginals of the true mechanical state s (blue) with those of the phantom state
890 $\hat{s}$ (orange) on a per-component basis: the five Efficiency channels at even indices coincide almost

perfectly, but the five flow capacity (W_c) channels at odd indices are systematically shifted upward in $\tilde{s}$, a direct empirical footprint of the phantom coupling Cc_t in equation 19. Figure 7 shows the per-module fouling marginals, bounded in $[0, 0.04]$ and exponentially decreasing due to the wash schedule. Figure 9 shows the episode-length distribution, which is capped at 20 000 steps and floored at 8 000 steps.

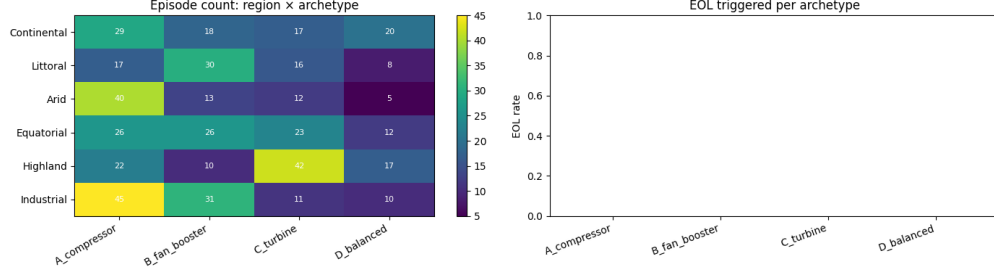


Figure 4: Region $\times$ archetype episode counts for the TurboSens2 train split, together with the EOL-triggered rate per archetype (identically zero in all splits, consistent with the proactive maintenance policy).

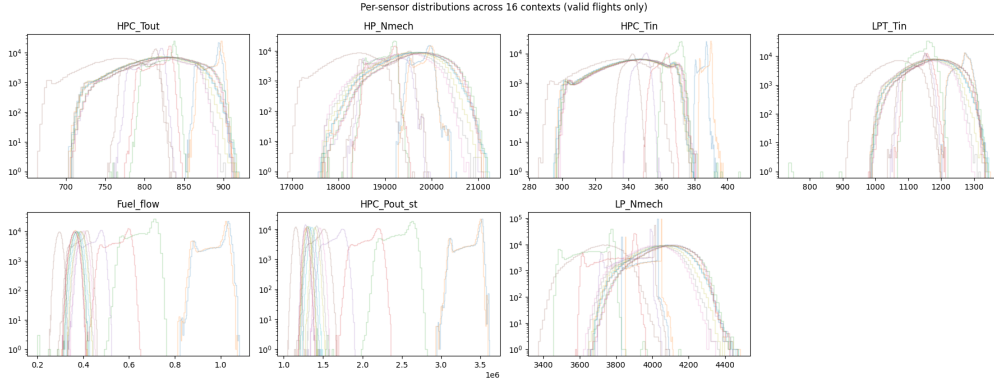


Figure 5: Marginal distribution of the seven DeckSMR sensors across the sixteen flight-phase contexts, TurboSens2 train split. Each subplot overlays the sixteen contexts for one sensor on a shared log-scale y-axis; the separation between curves reflects the context-dependent operating envelope exposed to the world model via ϕ_t .

D Architecture and Implementation Details

This section documents the encoder, predictor, optimisation, and probe configurations used to produce every JEPA and AR LSTM number reported in Section 5.1. The hyperparameters listed here are the defaults; per run deviations are recorded in Section E.1.

D.1 Sensor Encoder

A turbofan observation $\mathbf{o}_t \in \mathbb{R}^{n_s}$ with $n_s = S \cdot C$ is the concatenation of S sensor channels read across C flight phase contexts, all reflecting the same degradation state. TurboSens1 has $S=7$, $C=12$, $n_s=84$; TurboSens2 enlarges the context vector to include four operating point parameters per context, so $n_s = (7 + 4) \cdot 16 = 176$. Earlier iterations of this benchmark encoded $\mathbf{o}_t$ as a grayscale image and applied a Vision Transformer [?]; we replace that pipeline with a sensor native transformer that treats each of the n_s scalars as an independent token, removing the spatial inductive bias mismatch of the image based encoder.

The encoder $\text{Enc}_\theta : \mathbb{R}^{n_s} \rightarrow \mathbb{R}^d$ projects each scalar x_i to a d dimensional token via a shared linear layer, adds a learnable positional embedding $\mathbf{p}_i$, runs the sequence through an L layer Pre LayerNorm

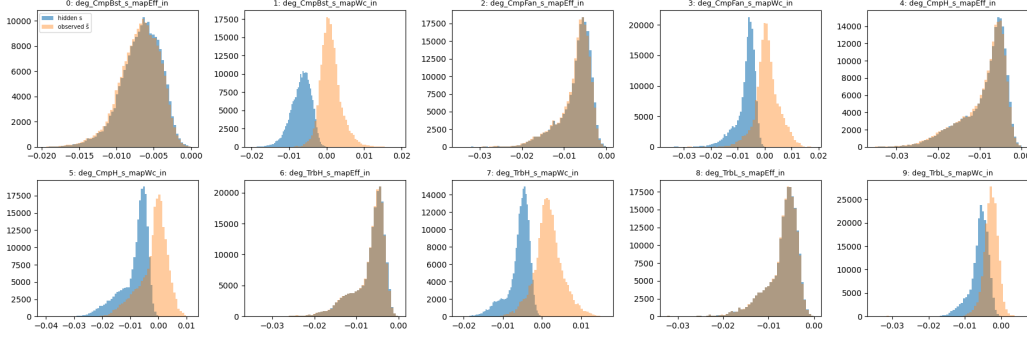


Figure 6: Per-component marginals of the true mechanical state s_t (blue) and the phantom state $\tilde{s}_t$ (orange), TurboSens2 train split. Efficiency channels at even indices overlap almost perfectly; flow capacity channels at odd indices are displaced upward in $\tilde{s}_t$ by the fouling-driven coupling Cc_t . The gap between the two histograms is exactly the non-identifiability signal the world model must learn to close by integrating the action history.

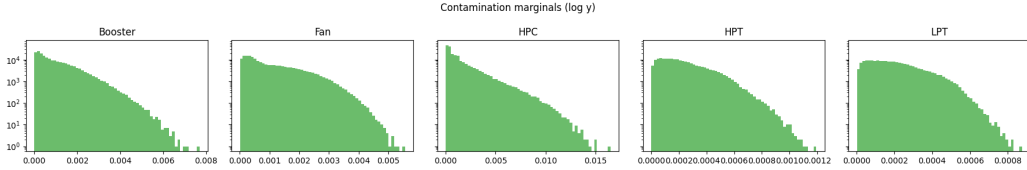


Figure 7: Marginal distribution of the five fouling channels (Booster, Fan, HPC, HPT, LPT), TurboSens2 train split. Log-scale y-axis. The HPC channel concentrates higher mass at large values, reflecting the combination of (i) higher base fouling rate and (ii) a sulfur-driven multiplier in the Industrial region.

910 Transformer encoder [?], and mean pools across tokens:

$$\mathbf{z}_t = \text{LayerNorm} \left(\frac{1}{n_s} \sum_{i=1}^{n_s} \text{TF}_{\theta}(\mathbf{W}_v x_{t,i} + \mathbf{p}_i) \right), \quad \mathbf{z}_t \in \mathbb{R}^d. \quad (24)$$

911 The encoder output dimension $d=64$ is matched to the world model latent dimension so the embedding
912 is fed to the predictor without an extra projection bottleneck. Table 18 summarises the configuration.

Table 18: Sensor encoder configuration.

Hyperparameter	Value
Input tokens n_s	84 (TurboSens1) or 176 (TurboSens2)
Embedding dimension d	64
Attention heads	4
Transformer layers L	2
Feed forward multiplier	$4\times$
Normalisation	Pre LayerNorm
Dropout	0.1
Positional encoding	Learnable
Pooling	Mean over tokens

913 D.2 Temporal Aggregator

914 The encoder in (24) reads a single snapshot, so it cannot directly express degradation rate or trend
915 direction. When the encoder is configured with a temporal window $w > 1$, each of the w consecutive
916 frames $\mathbf{o}_{t-w+1}, \dots, \mathbf{o}_t$ is encoded independently with shared weights, and a lightweight Temporal
917 Aggregator fuses the per frame embeddings into one trajectory aware embedding $\mathbf{z}_t^{\text{agg}} \in \mathbb{R}^d$. The

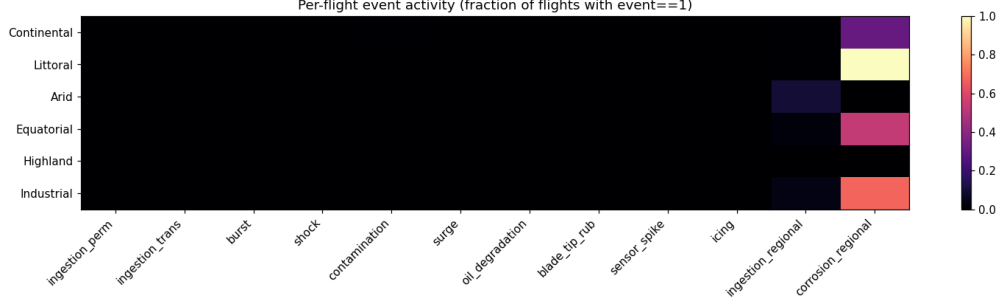


Figure 8: Per-flight event activity matrix (fraction of flights with any positive entry in the event channel), TurboSens2 train split. The two rightmost columns correspond to the regional-only events and dominate the picture: `regional_perm_1` is concentrated in Arid / Industrial / Equatorial, `regional_drift_1` in Littoral / Equatorial / Industrial. Note that the figure axis labels in this rendering still use the legacy code labels of the released simulator. Figures for the test and test-hard splits are qualitatively identical and available in the released figure pack.

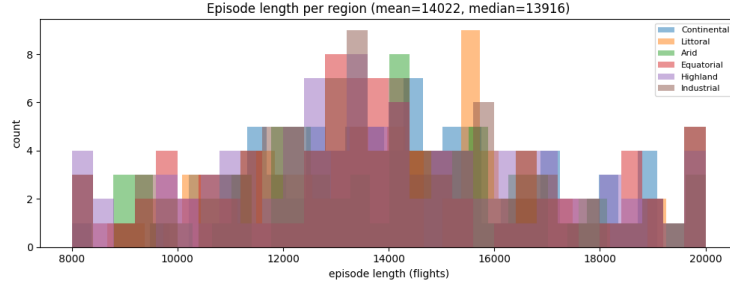


Figure 9: Episode length distribution, TurboSens2 train split. Lengths are bounded in $[8\,000, 20\,000]$ steps and centred at roughly 14 000 steps; episodes are terminated when the maintenance scheduler cannot keep the mechanical state above the proactive threshold.

918 aggregator is a single layer transformer encoder ($d=64$, 4 heads) with a learnable positional code
 919 per slot; the output at the last position attends over the full window and is used as the aggregated
 920 embedding:

$$\mathbf{z}_t^{\text{agg}} = \text{TempAgg}(\mathbf{z}_{t-w+1}, \dots, \mathbf{z}_t). \quad (25)$$

921 Setting $w=1$ recovers the snapshot baseline; the parameter overhead of the aggregator is $\mathcal{O}(d^2)$,
 922 negligible relative to the predictor.

923 D.3 World Model and Loss

924 The world model follows the Joint Embedding Predictive Architecture (JEPA, LeWM) [3, 9]. Given
 925 a context window of H consecutive embeddings $\mathbf{z}_{t-H+1}^{\text{agg}}, \dots, \mathbf{z}_t^{\text{agg}}$ and the corresponding action
 926 embeddings, an autoregressive predictor g_ψ outputs the next latent $\hat{\mathbf{z}}_{t+1}^{\text{agg}}$. Training combines a latent
 927 prediction loss with a SIGReg term [3] that encourages an isotropic Gaussian latent and prevents
 928 collapse:

$$\mathcal{L} = \underbrace{\|g_\psi(\mathbf{z}_{t-H:t}^{\text{agg}}, \mathbf{a}_{t-H:t}) - \mathbf{z}_{t+1}^{\text{agg}}\|_2^2}_{\text{prediction loss}} + \lambda \cdot \mathcal{L}_{\text{SIGReg}}. \quad (26)$$

929 D.4 Predictor and AR LSTM Baseline

930 The JEPA predictor g_ψ is a causal transformer (depth 4, 8 heads, MLP dimension 512, head dimension
 931 32, dropout 0.1) that ingests the history of H context embeddings and their action embeddings and
 932 emits one next step latent. A two layer MLP projector (BatchNorm) maps the predictor output back
 933 to the shared $d=64$ latent.

The AR LSTM baseline shares the sensor encoder and the temporal aggregator and replaces only the latent dynamics: the transformer predictor is swapped for a two layer LSTM (hidden size 256, dropout 0.1). All other components, including the SIGReg term and the projector, are identical, so any difference at evaluation time isolates the predictor family. The LSTM predictor has 1.14M parameters, comparable to the JEPA predictor.

D.5 Training Setup

All models train with AdamW ($\text{lr}=7 \times 10^{-5}$, weight decay 10^{-3}) under a linear warmup followed by cosine annealing, gradient clipping at global norm 0.3, and an effective batch size of 512 (achieved either as a native batch of 512 or as a smaller native batch with gradient accumulation; the effective batch is held constant across runs). The SIGReg regulariser uses $\lambda=0.15$, 17 knots, 1024 random projections.

Epoch budget. JEPA configurations are trained for 10 epochs, sufficient for the latent prediction loss to plateau on TurboSens2 at the effective batch size of 512; longer runs do not change the probe ranking. The AR LSTM baseline does not have to share the same epoch budget because its predictor family converges differently; we report it under a comparable wall budget for fairness, with per run details in Section E.1.

Hardware. Each run uses a single GPU, either an RTX 5090 or an H100. The hardware choice does not affect the reported numbers; differences between runs are explained by the hyperparameters listed in Section E.1.

Table 19: Default training hyperparameters used across JEPA and AR LSTM runs.

Hyperparameter	Value
Optimiser	AdamW
Learning rate	7×10^{-5}
Weight decay	10^{-3}
LR schedule	Linear warmup + cosine annealing
Gradient clip (global norm)	0.3
Effective batch size	512
Epochs (JEPA)	10
SIGReg weight λ	0.15
SIGReg knots / projections	17 / 1024
Predictor history H	{8, 16, 32}
Encoder window w	{1, 5, 10}
Predictor stride S	{1, 5, 10, 20}

D.6 Downstream Evaluation Probes

After training, the encoder and the temporal aggregator (when $w > 1$) are frozen, and small heads are trained on the frozen embeddings to expose what the world model representation actually captures. The probes use the same train and test splits as the world model, but they see the ground truth target as a supervised signal that the world model itself never accesses during pretraining.

Health state probe. The main probe maps a frozen embedding to the ten dimensional ground truth health state s_t . We use a small transformer regression head (2 layers, hidden dimension 128, 4 heads), trained with Adam ($\text{lr}=10^{-3}$) for up to 150 epochs with early stopping on mean per dimension RMSE. We report per component R^2 , RMSE, and Pearson r . Two probe input contexts are reported alongside each encoder, to separate encoder quality from probe expressivity:

- **single embedding:** each input is one frozen embedding; the probe head reduces to an MLP and reflects what the encoder alone has captured;
- **trajectory window:** each input is a window of 10 consecutive embeddings, so the probe transformer can attend over the trajectory. Differences against the single embedding score show how much temporal context the encoder has already absorbed.

Action timing probe. A second, optional probe predicts the number of steps to the next maintenance action, $\tau_t = \min(\text{steps to next maintenance}, \tau_{\max})$, with $\tau_{\max}=300$. Targets are $\log(1 + \tau_t)$ transformed before standardisation; metrics are computed on uncensored timesteps only. Across all reported encoders the linear probe Pearson is at or below 0.03, indicating that 10 epoch self supervised training does not expose maintenance timing structure to a linear readout. We report this probe as a placeholder for future work that should explicitly handle event sparsity.

What each temporal hyperparameter controls. The encoder window w , the predictor history H , and the probe input length `seq_len` are independent design knobs whose effects are summarised in Table 20.

Parameter	Affects encoder?	Affects predictor?	Affects probe?
w (encoder window)	Yes, depth of temporal fusion inside $\mathbf{z}_t$	Indirectly, through richer $\mathbf{z}_t$	Indirectly
H (predictor history)	No	Yes, number of past embeddings the predictor sees	No
<code>seq_len</code>	No	No	Yes, window fed to the probe head

Table 20: Effect of each temporal hyperparameter.

E TurboSens2 Sweep Campaign Details

This appendix records the per run accounting and the per HI breakdown referenced from Section 5.1.

E.1 Run inventory

Table 21 lists the JEPA runs. All runs use $P=4$, effective batch size 512, AdamW with cosine schedule and learning rate $7 \cdot 10^{-5}$, and the `hi_probe` callback firing at epochs 5 and 9. Activations rather than weights dominate VRAM, which is why E_3 at $S=5$ requires a native batch of 128 with gradient accumulation of four to keep the effective batch fixed.

Table 21: JEPA runs (TurboSens2 train split, 10 epochs). Final HI mean Pearson is reported as multi seed mean $\pm$ std where seeds were available. E_4 ($H=32$, $S=5$) is paused; E_7 ($H=16$, $S=20$) is queued.

ID	H	S	P	wall	fit/loss	seeds	test r	test_hard r
E_1	16	1	4	4.4h	0.406	2	0.447 ± 0.033	0.053 ± 0.138
E_2	32	1	4	8.2h	0.254	2/3	0.588 ± 0.028	0.262 ± 0.073
E_3	16	5	4	20h	0.259	2	0.431 ± 0.032	0.340 ± 0.041
E_5	8	1	4	—	—	2	0.514 ± 0.007	-0.028 ± 0.092
E_6	16	10	4	—	—	2/3	0.469 ± 0.153	0.369 ± 0.052
E_4	32	5	4	—	—	—	paused	paused
E_7	16	20	4	—	—	—	queued	queued

Table 22 lists the AR LSTM runs (1.14M predictor parameters). Final mean Pearson on the in distribution `test` split and the OOD `test_hard` split is also given here for reference; these are the same numbers plotted in Figure 2. Configurations L_3 and L_4 are deferred for the same reason as E_4 : the specificity vs generality direction they would test is already established by the JEPA $S=5$ run.

Table 22: AR LSTM runs (TurboSens2 train split, 10 epochs).

ID	H	S	P	native bs / accum	wall	fit/loss	test r	test_hard r
L_1	16	1	4	512 / 1	7h	0.379	0.564	0.325
L_2	32	1	4	256 / 2	13h	0.234	0.495	-0.014
L_3	16	5	4	128 / 4	—	—	—	—
L_4	32	5	4	64 / 8	—	—	—	—

988 E.2 VRAM scaling and effective sequence length

989 The encoder consumes the full window of raw frames before the strided subsampling step, so VRAM
 990 scales with the number of raw timesteps $n_{\text{steps}} = (H + P - 1) \cdot S + W$ rather than with the strided
 991 token count. An empirical fit on a 32 GB GPU gives $\text{VRAM (GB)} \approx 0.00146 \cdot \text{bs} \cdot n_{\text{steps}}$; the E_3 and
 992 E_4 rows in Table 21 are sized against this fit with a 28 GB safety budget. A practical consequence is
 993 that the $S=5$ runs are the most expensive in the sweep despite having the same strided history length
 994 as $S=1$.

995 E.3 Per HI breakdown of E_2 and E_1

996 Figure 10 reports per component Pearson (top) and RMSE (bottom) for the in distribution leader E_2
 997 on both splits. Three dimensions sign flip on OOD (HI_1 , HI_2 , HI_3): the in distribution probe is anti
 998 correlated with the ground truth on the held out split for these components. The Pearson and RMSE
 999 rows disagree on which dimensions are well recovered: HI_2 has an acceptable OOD Pearson but
 1000 the worst RMSE, while HI_8 has a high in distribution Pearson and a moderate one on OOD. Single
 1001 metric reporting hides this disagreement.

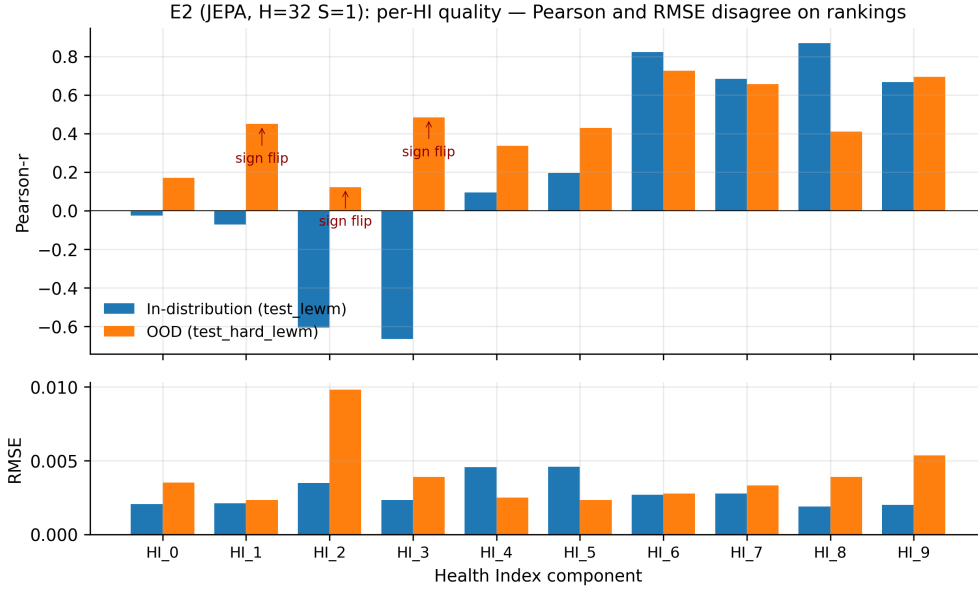


Figure 10: Per component Pearson (top) and RMSE (bottom) for E_2 ($H=32$, $S=1$) on test (blue) and test_hard (orange). Three sign flips on OOD; HI_2 has the worst RMSE despite an acceptable Pearson. Pearson and RMSE rank components differently.

1002 Table 23 reports the per dimension Pearson and relative drop for the E_1 encoder on a single seed
 1003 (seed=0), the same data plotted in Figure ???. The released eval pipeline currently aggregates only
 1004 the mean Pearson across seeds; per dimension Pearson and RMSE will be added to the SUMMARY
 1005 in the next release and the multi seed columns of Table ??? will be filled in. HI_3 is the only dimension
 1006 recovered nearly equally on both splits; HI_8 flips sign, indicating that the encoder learned a coupling
 1007 that holds in the training draw but inverts on a different seed batch with a higher rate of extreme
 1008 episodes. Most dimensions sit on a continuum between these extremes.

1009 E.4 Probe over single embedding versus full history

1010 The probe head used in Section 5.1 reports seq_len=1 as default, mapping a single embedding $\mathbf{z}_t$
 1011 to $\mathbf{s}_t$. The released eval pipeline also supports temporally aggregated probes at $\text{seq_len} \in \{10, 50\}$.
 1012 Single seed numbers for the AR LSTM checkpoints L_1 ($H=16$, $S=1$) and L_{big} ($H=32$, $S=1$, $W=4$)
 1013 are summarised in Table 24; multi seed bands at seq_len=10 and 50 are pending. The qualitative
 1014 picture across these single seed measurements is that seq_len=10 helps slightly on the in distribution

Table 23: Per HI Pearson breakdown for E_1 ($H=16$, $S=1$, TurboSens2, single seed). Negative relative deltas indicate OOD collapse; HI_8 flips sign.

HI dimension	test Pearson	test_hard Pearson	relative drop
HI_0	+0.43	+0.02	−94%
HI_1	+0.57	+0.07	−87%
HI_2	+0.51	+0.12	−76%
HI_3	+0.49	+0.42	−14%
HI_4	+0.51	+0.17	−67%
HI_5	+0.34	+0.19	−44%
HI_6	+0.73	+0.23	−68%
HI_7	+0.63	+0.20	−68%
HI_8	+0.71	−0.27	−138% (sign flip)
HI_9	+0.72	+0.36	−50%
mean	+0.563	+0.152	−73%

split and modestly on the OOD split for L_{big} but does not move the headline ranking; we recommend that future TurboSens evaluations report at least `seq_len=1` and `seq_len=10` alongside.

Table 24: HI Pearson by probe `seq_len` for two AR LSTM checkpoints. Single seed (`seed=0`); multi seed bands at `seq_len=10, 50` are pending a rerun.

ckpt	test			test_hard		
	sl=1	sl=10	sl=50	sl=1	sl=10	sl=50
L_1 ($H=16$, $S=1$, $W=1$)	0.564	0.510	0.549	0.325	0.378	0.241
L_{big} ($H=32$, $S=1$, $W=4$)	0.559	0.555	0.559	0.329	0.243	0.051

E.5 Probe data size sensitivity

The default in training probe sampled 28k embedding timesteps and reported a mean Pearson of 0.25 for both the JEPA E_1 and the AR LSTM L_1 encoder. Re evaluating the same encoders with the same linear head on the full $\sim 770k$ step training stream (`eval_sweep`) raised the in distribution Pearson to 0.56 for both, a $\times 2.3$ jump from a probe data budget change alone. An eight times wider probe head on the small budget did not close the gap and in some runs hurt the score, ruling out probe head capacity as the limiting factor. We therefore fix the larger budget for all numbers reported in Section 5.1.

E.6 Counterfactual fidelity curves

Figure 11 reports the per checkpoint and per action curves underlying the counterfactual fidelity summary in Section 5.1. The top row plots absolute mean HI RMSE between the world model and the simulator under the same overridden action; the bottom row plots the differential $|\Delta_{\text{model}}(a, 0) - \Delta_{\text{sim}}(a, 0)|$, which subtracts the *do nothing* baseline from both rollouts and isolates the action specific response. Curves are means over thirty episodes and three branch positions per checkpoint, with a one standard deviation band around the mean.

E.7 Cross split scatter

Figure 12 places each (architecture, config) pair on the in distribution vs OOD plane. The dashed identity line marks perfect generalisation. JEPA E_2 and E_3 sit above the line (better OOD than in distribution), the AR LSTM L_1 sits closest to the identity (most balanced), and E_1 , L_2 sit below the line (specialised to in distribution). The figure makes the specificity vs generality axis legible at a glance and is complementary to the bar chart in Figure 2.

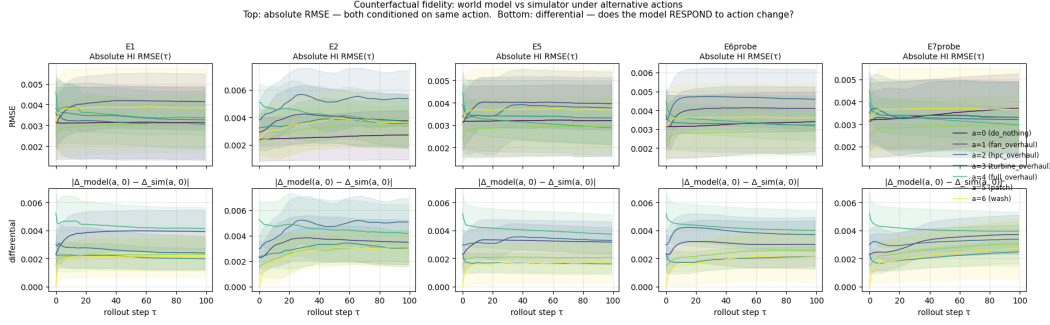


Figure 11: Counterfactual fidelity per checkpoint and per action. Top row: absolute HI RMSE between world model and simulator under the same overridden action a . Bottom row: differential $|\Delta_{\text{model}}(a, 0) - \Delta_{\text{sim}}(a, 0)|$, isolating the response to the action change against the *do nothing* baseline.

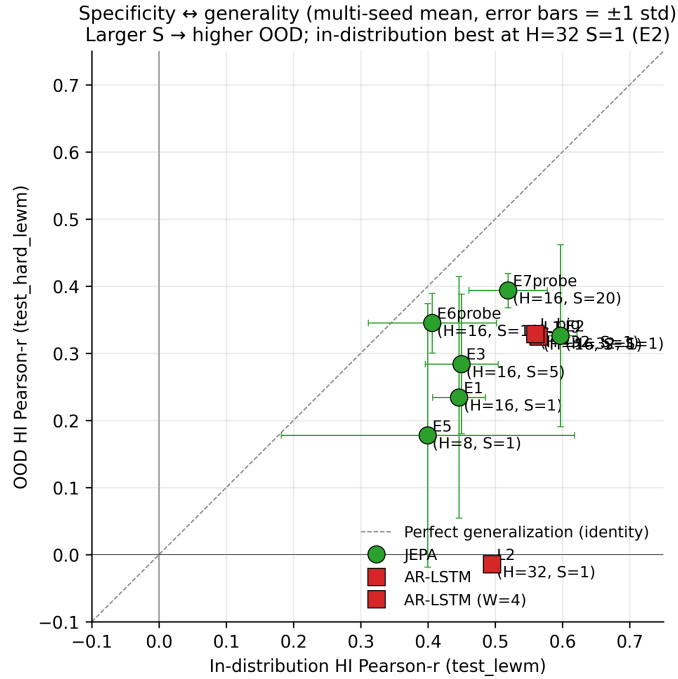


Figure 12: In distribution vs OOD HI Pearson for the five evaluated configurations. JEPA in green circles, AR LSTM in red squares. Dashed identity line marks perfect generalisation; points above the line generalise better than they specialise.

1038 F Auxiliary Evaluation Tasks

1039 The benchmark ships with two further tasks that are not the focus of the present paper but are useful
 1040 for stress testing world model representations in future work. The data and metric infrastructure for
 1041 both tasks is included in the released evaluation code.

1042 F.1 Degradation velocity estimation

1043 Rather than predicting the absolute health state, a velocity probe asks whether the encoder captures
 1044 the local rate of change of degradation. The target is $\Delta s_t = s_t - s_{t-1}$ within episode boundaries
 1045 and restricted to non maintenance timesteps. Predicting Δs_t from a single snapshot is strictly
 1046 harder than predicting s_t : it requires the encoder to capture local curvature of the degradation

1047 manifold. A maintenance alarm score is defined by thresholding $\|\widehat{\Delta s}_t\|_2$ and evaluated at horizons
1048 $K \in \{10, 20, 50, 100\}$ as the AUC of the binary event “maintenance occurs within K steps”.

1049 **F.2 Deterministic replay and counterfactual API**

1050 The `scenarios.scenario4.replay.EpisodeReplayer` class is a pure python entry point around
1051 the simulator. It takes a generated HDF5 file, reads the `ep_meta/seed`, `region` and `is_extreme`
1052 flag of any episode, and either replays the full trajectory bit identically or forks it at a chosen
1053 branch point. The data class `CounterfactualSpec` exposes four levers: `branch_t` (where to
1054 fork), `override_action` (closed loop, the expert maintenance policy resumes after the forced
1055 action), `forced_actions` (open loop, a fixed sequence of actions is applied after `branch_t`), and
1056 `weather_seed + weather_overrides` (re seed the seasonal OU). Bit identity is enforced by a
1057 regression test (`test/scenario4/test_replay_bit_identity.py`) and the schema requirement
1058 is documented in the file’s docstring; the source tree currently lives on the `nodegpu` branch.

1059 **F.3 Latent forecasting with action divergence gap**

1060 The world model’s latent predictor is evaluated directly by rolling out τ steps autoregressively
1061 from an initial context window and decoding each predicted embedding $\hat{z}_{t+\tau}$ to HI space using the
1062 Task 1 probe (no additional training). Rollouts use zero actions; the world model propagates the
1063 current degradation trajectory without any maintenance intervention. To quantify the cost of ignoring
1064 maintenance, each τ step trajectory is split into clean segments (no maintenance event in $(t, t+\tau]$)
1065 and event segments (at least one maintenance event); we report mean HI RMSE on both and their
1066 difference $\delta(\tau) = \text{RMSE}_{\text{event}}(\tau) - \text{RMSE}_{\text{clean}}(\tau)$, the action divergence gap. A large gap at short
1067 horizons indicates that maintenance transitions are the primary source of forecast error and motivates
1068 action conditioned world modelling. The action conditioned counterfactual rollout that closes this
1069 gap is the camera ready experimental target referenced in Section 5.1.

1070 **G Illustrative Trajectories and Dataset Views**

1071 This section collects example renderings from the released simulator’s EDA notebooks, intended to
1072 give the reader a qualitative sense of what TurboSens trajectories and observations look like. None of
1073 these figures is part of the evaluation protocol; they are illustrative. Some figures still carry the legacy
1074 event labels (e.g. `shock`, `compressor_surge`, `ingestion_perm`) on their axes; these correspond
1075 to the abstract effect typed labels used in the rest of the paper.

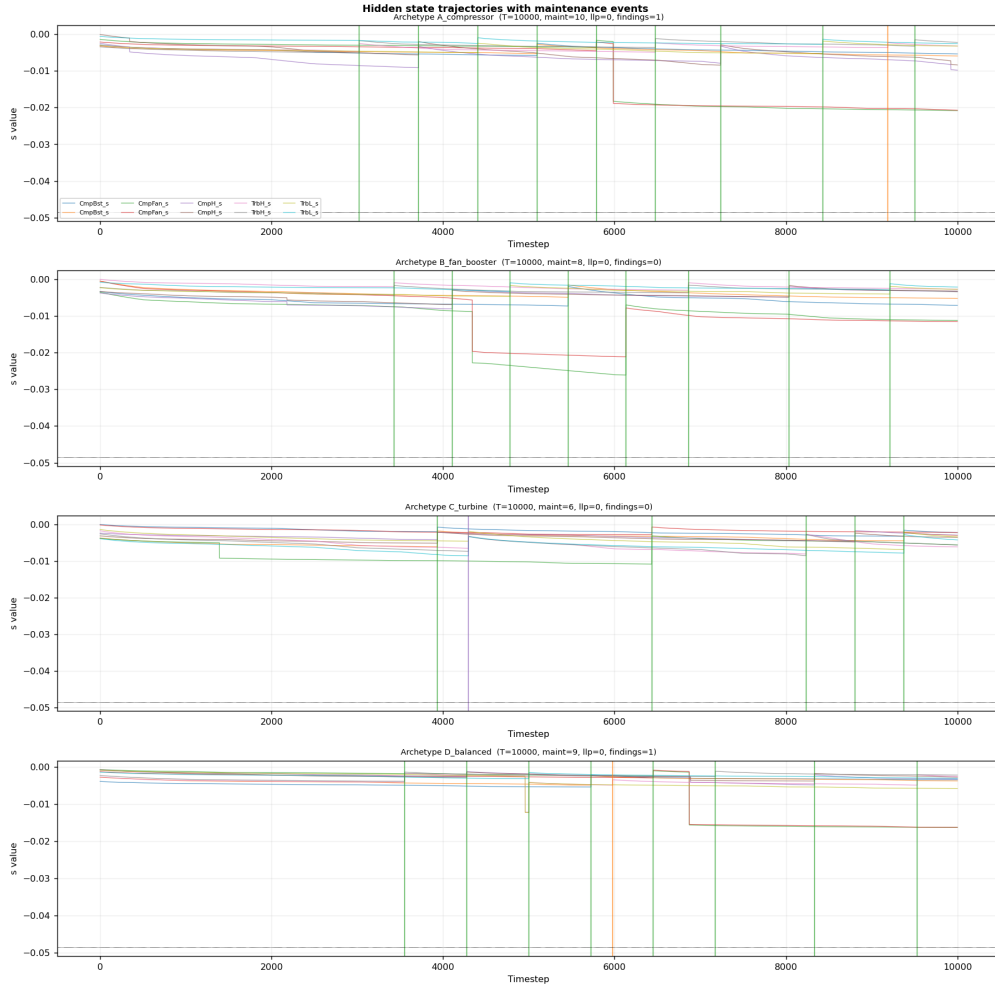


Figure 13: TurboSens1 hidden state trajectories under the four degradation archetypes (A compressor, B fan/booster, C turbine, D balanced). Each panel overlays the ten state components for a single representative episode; vertical green and orange lines mark maintenance interventions. Archetype D produces nearly flat trajectories that survive the full 10,000 step horizon without intervention, while A and C show the steeper early drift that drives recurring maintenance.

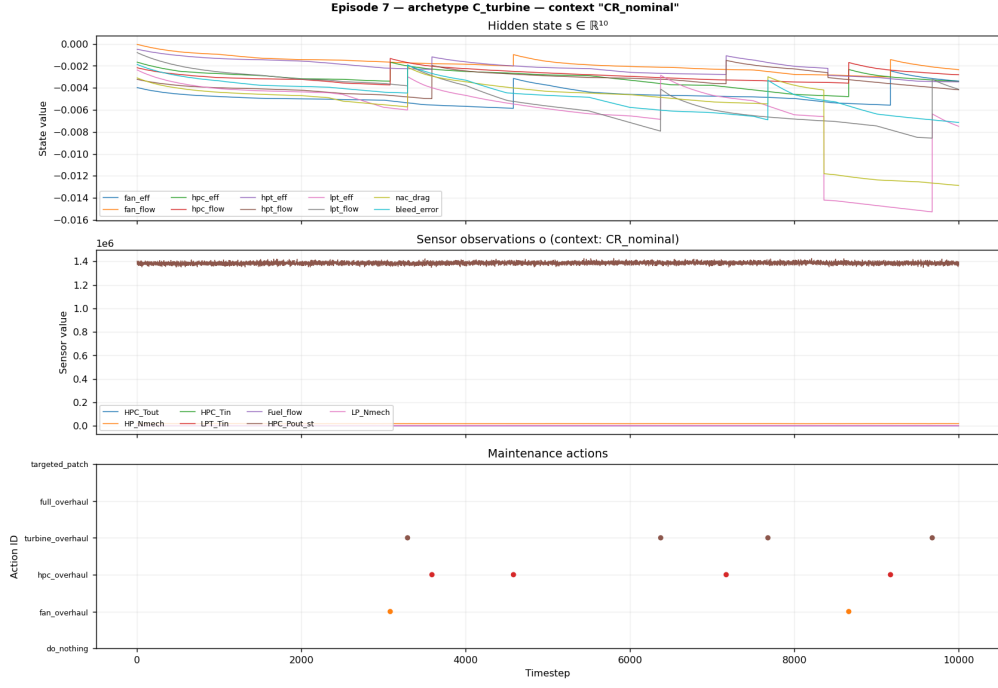


Figure 14: Single TurboSens1 episode (archetype C, observed under the CR_nominal flight context). Top: the ten dimensional hidden state s_t evolving as a sawtooth of partial resets. Middle: the seven dimensional sensor stream o_t at the chosen context, showing how observation noise and contextual variation obscure the underlying state. Bottom: the discrete maintenance action sequence; coloured markers indicate the action type at each maintenance event.

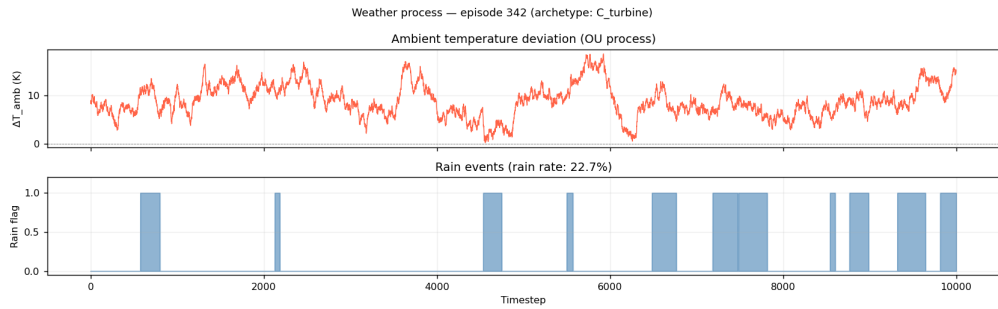


Figure 15: Weather process for a single TurboSens1 episode: the ambient temperature deviation ΔT_{amb} follows a slow Ornstein-Uhlenbeck trajectory (top) and rain events (bottom) are gated by the seasonal phase, here at a 22.7% episode wide rain rate.

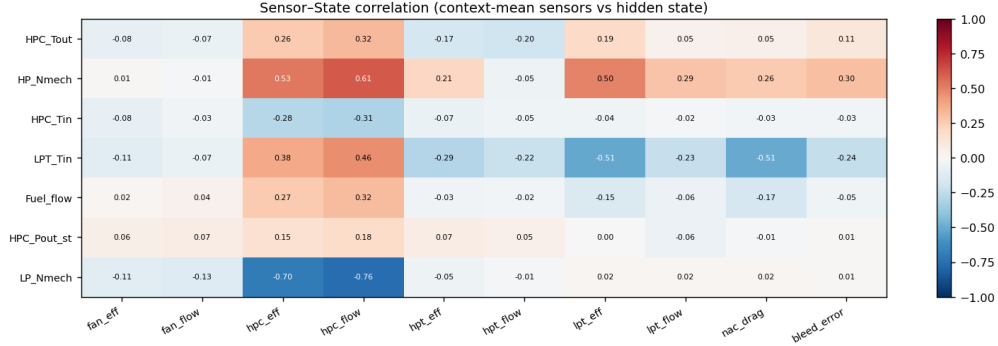


Figure 16: TurboSens1 sensor / state correlation matrix. Context averaged Pearson correlations between the seven sensor channels and the ten hidden state dimensions, computed over the train split. Most cells are near zero, but a handful of strong positive (HP shaft speed vs HPC efficiency / flow, $r \in [0.5, 0.6]$) and strong negative (LP shaft speed vs HPC flow, $r = -0.76$) couplings establish that the observation map is informative about the latent state but far from a one-to-one decoder; this is the structure the inverse evaluation protocol asks the encoder to recover.

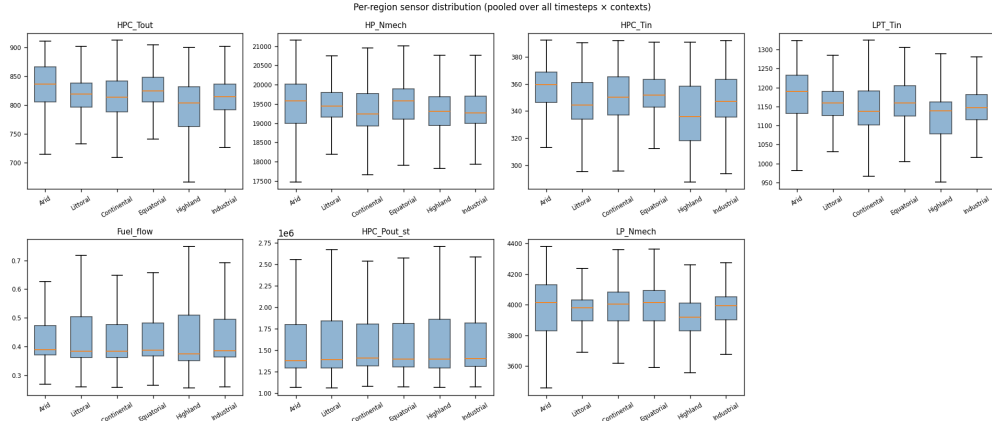


Figure 17: TurboSens2 per region sensor distributions, pooled over all timesteps and contexts. Each panel shows a box plot of one of the seven sensor channels across the six geographic regions (Arid, Littoral, Continental, Equatorial, Highland, Industrial). Region affects both the median and the spread of every channel, most noticeably on HPC inlet and outlet temperatures (HPC_Tin, HPC_Tout) where the Highland and Arid extremes are visibly separated. The OOD evaluation in Section 5 relies on held out region / archetype draws sitting in these distributional tails.

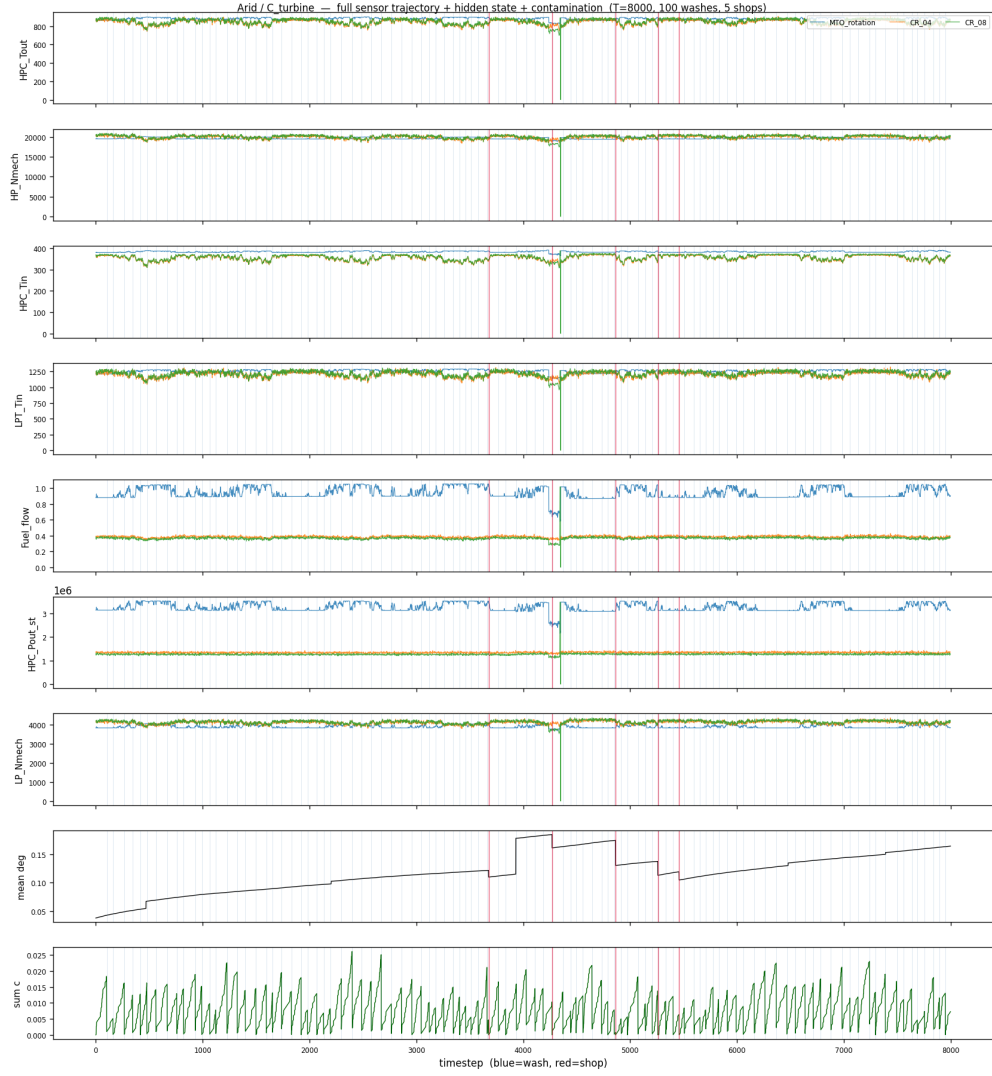


Figure 18: Full trajectory of a TurboSens2 episode in the Arid region (archetype C, $T \approx 8000$ steps, ~ 100 engine washes, 5 shop visits). Each row shows one of the seven sensors over the full episode. Vertical red and orange markers indicate shop visits and engine washes respectively; the dramatic dips in HP_Nmech and HPC_Pout_st around the shop visits are visible directly in the sensor stream. The bottom two rows show the calendar month and a fouling proxy, illustrating how often the wash action is exercised in a high fouling region.

1076 NeurIPS Paper Checklist

1077 The checklist is designed to encourage best practices for responsible machine learning research,
1078 addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove
1079 the checklist: **The papers not including the checklist will be desk rejected.** The checklist should
1080 follow the references and follow the (optional) supplemental material. The checklist does NOT count
1081 towards the page limit.

1082 Please read the checklist guidelines carefully for information on how to answer these questions. For
1083 each question in the checklist:

- 1084 • You should answer [Yes], [No], or [N/A].
- 1085 • [N/A] means either that the question is Not Applicable for that particular paper or the
1086 relevant information is Not Available.
- 1087 • Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

1088 **The checklist answers are an integral part of your paper submission.** They are visible to the
1089 reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it
1090 (after eventual revisions) with the final version of your paper, and its final version will be published
1091 with the paper.

1092 The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation.
1093 While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a
1094 proper justification is given (e.g., error bars are not reported because it would be too computationally
1095 expensive” or “we were unable to find the license for the dataset we used”). In general, answering
1096 [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we
1097 acknowledge that the true answer is often more nuanced, so please just use your best judgment and
1098 write a justification to elaborate. All supporting evidence can appear either in the main paper or the
1099 supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification
1100 please point to the section(s) where related material for the question can be found.

1101 IMPORTANT, please:

- 1102 • **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- 1103 • **Keep the checklist subsection headings, questions/answers and guidelines below.**
- 1104 • **Do not modify the questions and only use the provided macros for your answers.**

1105 1. Claims

1106 Question: Do the main claims made in the abstract and introduction accurately reflect the
1107 paper’s contributions and scope?

1108 Answer: **[TODO]**

1109 Justification: **[TODO]**

1110 Guidelines:

- 1111 • The answer [N/A] means that the abstract and introduction do not include the claims
1112 made in the paper.
- 1113 • The abstract and/or introduction should clearly state the claims made, including the
1114 contributions made in the paper and important assumptions and limitations. A [No] or
1115 [N/A] answer to this question will not be perceived well by the reviewers.
- 1116 • The claims made should match theoretical and experimental results, and reflect how
1117 much the results can be expected to generalize to other settings.
- 1118 • It is fine to include aspirational goals as motivation as long as it is clear that these goals
1119 are not attained by the paper.

1120 2. Limitations

1121 Question: Does the paper discuss the limitations of the work performed by the authors?

1122 Answer: **[TODO]**

1123 Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- 1229 • Providing as much information as possible in supplemental material (appended to the
1230 paper) is recommended, but including URLs to data and code is permitted.

1231 **6. Experimental setting/details**

1232 Question: Does the paper specify all the training and test details (e.g., data splits, hyperpa-
1233 rameters, how they were chosen, type of optimizer) necessary to understand the results?

1234 Answer: **[TODO]**

1235 Justification: **[TODO]**

1236 Guidelines:

- 1237 • The answer **[N/A]** means that the paper does not include experiments.
- 1238 • The experimental setting should be presented in the core of the paper to a level of detail
1239 that is necessary to appreciate the results and make sense of them.
- 1240 • The full details can be provided either with the code, in appendix, or as supplemental
1241 material.

1242 **7. Experiment statistical significance**

1243 Question: Does the paper report error bars suitably and correctly defined or other appropriate
1244 information about the statistical significance of the experiments?

1245 Answer: **[TODO]**

1246 Justification: **[TODO]**

1247 Guidelines:

- 1248 • The answer **[N/A]** means that the paper does not include experiments.
- 1249 • The authors should answer **[Yes]** if the results are accompanied by error bars, confidence
1250 intervals, or statistical significance tests, at least for the experiments that support the
1251 main claims of the paper.
- 1252 • The factors of variability that the error bars are capturing should be clearly stated (for
1253 example, train/test split, initialization, random drawing of some parameter, or overall
1254 run with given experimental conditions).
- 1255 • The method for calculating the error bars should be explained (closed form formula,
1256 call to a library function, bootstrap, etc.)
- 1257 • The assumptions made should be given (e.g., Normally distributed errors).
- 1258 • It should be clear whether the error bar is the standard deviation or the standard error
1259 of the mean.
- 1260 • It is OK to report 1-sigma error bars, but one should state it. The authors should
1261 preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis
1262 of Normality of errors is not verified.
- 1263 • For asymmetric distributions, the authors should be careful not to show in tables or
1264 figures symmetric error bars that would yield results that are out of range (e.g., negative
1265 error rates).
- 1266 • If error bars are reported in tables or plots, the authors should explain in the text how
1267 they were calculated and reference the corresponding figures or tables in the text.

1268 **8. Experiments compute resources**

1269 Question: For each experiment, does the paper provide sufficient information on the com-
1270 puter resources (type of compute workers, memory, time of execution) needed to reproduce
1271 the experiments?

1272 Answer: **[TODO]**

1273 Justification: **[TODO]**

1274 Guidelines:

- 1275 • The answer **[N/A]** means that the paper does not include experiments.
- 1276 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
1277 or cloud provider, including relevant memory and storage.
- 1278 • The paper should provide the amount of compute required for each of the individual
1279 experimental runs as well as estimate the total compute.

- 1280 • The paper should disclose whether the full research project required more compute
1281 than the experiments reported in the paper (e.g., preliminary or failed experiments that
1282 didn't make it into the paper).

1283 **9. Code of ethics**

1284 Question: Does the research conducted in the paper conform, in every respect, with the
1285 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

1286 Answer: **[TODO]**

1287 Justification: **[TODO]**

1288 Guidelines:

- 1289 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
1290 Ethics.
1291 • If the authors answer [No], they should explain the special circumstances that require a
1292 deviation from the Code of Ethics.
1293 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
1294 eration due to laws or regulations in their jurisdiction).

1295 **10. Broader impacts**

1296 Question: Does the paper discuss both potential positive societal impacts and negative
1297 societal impacts of the work performed?

1298 Answer: **[TODO]**

1299 Justification: **[TODO]**

1300 Guidelines:

- 1301 • The answer [N/A] means that there is no societal impact of the work performed.
1302 • If the authors answer [N/A] or [No], they should explain why their work has no societal
1303 impact or why the paper does not address societal impact.
1304 • Examples of negative societal impacts include potential malicious or unintended uses
1305 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
1306 (e.g., deployment of technologies that could make decisions that unfairly impact specific
1307 groups), privacy considerations, and security considerations.
1308 • The conference expects that many papers will be foundational research and not tied
1309 to particular applications, let alone deployments. However, if there is a direct path to
1310 any negative applications, the authors should point it out. For example, it is legitimate
1311 to point out that an improvement in the quality of generative models could be used to
1312 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
1313 that a generic algorithm for optimizing neural networks could enable people to train
1314 models that generate Deepfakes faster.
1315 • The authors should consider possible harms that could arise when the technology is
1316 being used as intended and functioning correctly, harms that could arise when the
1317 technology is being used as intended but gives incorrect results, and harms following
1318 from (intentional or unintentional) misuse of the technology.
1319 • If there are negative societal impacts, the authors could also discuss possible mitigation
1320 strategies (e.g., gated release of models, providing defenses in addition to attacks,
1321 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
1322 feedback over time, improving the efficiency and accessibility of ML).

1323 **11. Safeguards**

1324 Question: Does the paper describe safeguards that have been put in place for responsible
1325 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
1326 image generators, or scraped datasets)?

1327 Answer: **[TODO]**

1328 Justification: **[TODO]**

1329 Guidelines:

- 1330 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

1382 Guidelines:

1383 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1384 with human subjects.

1385 • Including this information in the supplemental material is fine, but if the main contribu-

1386 tion of the paper involves human subjects, then as much detail as possible should be

1387 included in the main paper.

1388 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,

1389 or other labor should be paid at least the minimum wage in the country of the data

1390 collector.

1391 **15. Institutional review board (IRB) approvals or equivalent for research with human**

1392 **subjects**

1393 Question: Does the paper describe potential risks incurred by study participants, whether

1394 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)

1395 approvals (or an equivalent approval/review based on the requirements of your country or

1396 institution) were obtained?

1397 Answer: [TODO]

1398 Justification: [TODO]

1399 Guidelines:

1400 • The answer [N/A] means that the paper does not involve crowdsourcing nor research

1401 with human subjects.

1402 • Depending on the country in which research is conducted, IRB approval (or equivalent)

1403 may be required for any human subjects research. If you obtained IRB approval, you

1404 should clearly state this in the paper.

1405 • We recognize that the procedures for this may vary significantly between institutions

1406 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the

1407 guidelines for their institution.

1408 • For initial submissions, do not include any information that would break anonymity (if

1409 applicable), such as the institution conducting the review.

1410 **16. Declaration of LLM usage**

1411 Question: Does the paper describe the usage of LLMs if it is an important, original, or

1412 non-standard component of the core methods in this research? Note that if the LLM is used

1413 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

1414 scientific rigor, or originality of the research, declaration is not required.

1415 Answer: [TODO]

1416 Justification: [TODO]

1417 Guidelines:

1418 • The answer [N/A] means that the core method development in this research does not

1419 involve LLMs as any important, original, or non-standard components.

1420 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not

1421 be described.